arXiv:2610.17528v2 [cs.DC] 22 Oct 2026

ATTUNE: AGENTIC THROUGHPUT TUNING FOR LLM TRAINING

Leo Fan Hongyang Wang Wenbo Chen Jingguo Liu Xian Li Xuan Wang Ruochen Wen Yiming Yan
Hao Wu Yichuan Li Qian Qu Zheshen Wang Benoit Dumoulin
Amazon

ABSTRACT

The throughput of an LLM training job depends on its recipe: how the model is split across GPUs, how memory is traded for compute, and which kernels and communication overlaps it uses. The best recipe changes with every model, accelerator and framework version. Automated tuners search a fixed space of knobs and check only that a job runs, so they miss levers outside that space and can ship a configuration that silently changes training. ATTUNE is a coding agent that tunes any working recipe unattended within a GPU-hour budget. Guided by a skill of search rules, scripts and lessons from past tuning campaigns, it diagnoses each run and changes one lever at a time. A correctness gate outside the agent refuses any argument known to change the training objective. ATTUNE tunes Amazon's production post-training recipes on H100, H200 and B200 clusters, raising throughput by 1.7× on average. It beats the published throughput of four of NVIDIA's own benchmark recipes on NVIDIA's AutoModel framework by 6–17%. On models held out from its development, it improves the strongest public recipes most where they were written for another setting, by up to 2.6× on MiniMax-M2, and least where the vendor tuned them for the same hardware. Given the same GPU-hour budget, it outperforms existing automated tuners, including Bayesian optimization, random search and OptiCo, a recent LLM-agent tuner. Its gain on GLM-4.5 is more than four times the best alternative's. The same coding agent without ATTUNE's skill gains less and fails half its launches. We release ATTUNE at `github.com/amazon-science/attune` and OpenRecipes, a leaderboard of tuned and verified training recipes, at `openrecipes.ai`.

1 INTRODUCTION

Training large language models is expensive, and much of the cost depends on how efficiently each job uses its hardware. Reaching high efficiency requires a well-tuned training *recipe*, the full configuration a job runs with, from how the model is parallelized across GPUs (Shoeybi et al., 2019; Narayanan et al., 2021; Rajbhandari et al., 2022; Liu et al., 2025) to how memory, communication and kernels are managed. What works best depends on the model, the accelerator, the software stack and the data at hand. A recipe that works well in one setting rarely works as well in another. Finding a good recipe remains largely manual: engineers launch trial jobs, profile them to find the bottleneck and adjust the configuration until throughput stops improving (DeepSpeed Team, 2021; Jiang et al., 2024b). Well-tuned recipes are therefore published for only a few popular models and accelerators, and most teams must repeat this work for their own setting.

Existing tools automate parts of this tuning within a space fixed in advance. Parallelization planners (Zheng et al., 2022; Miao et al., 2022; Liu et al., 2024; Um et al., 2024) and vendor autoconfigurators (DeepSpeed Team, 2021; NVIDIA, 2026c) search parallel layouts, micro-batch sizes and memory knobs with a cost model, a simulator or a sweep. Black-box search such as Bayesian optimization (Bergstra et al., 2011) needs no model of the system but many costly runs. Recent LLM-agent tuners (Chen et al., 2026; Ding et al., 2025, 2026b) let agents read profiles and propose configurations, but over a fixed schema of knobs.

Two observations motivate a different design. First, the throughput a working recipe misses lies partly in levers that pay off only when changed together and partly in levers a fixed schema does not list, such as virtual pipeline stages or a different expert dispatcher. Finding it requires a tuner that diagnoses what limits each run, as an expert does, and can change any argument the framework exposes. Second, a configuration can change what the model trains without failing. An expert capacity factor, a standard throughput argument for mixture-of-experts (MoE) models, drops the tokens that overflow an expert. The flag `--grad-reduce-in-bf16` reads like a communication flag but also disables FP32 gradient accumulation. A tuner that checks only that a job runs could ship either.

Correspondence to: Leo Fan <jianhfan@amazon.com, leofanxyz@gmail.com>.

Preprint.

Coding agents make such a tuner practical. An agent such as Claude Code (Anthropic, 2026) reads source code, runs commands and works on one task for hours, the abilities an expert uses to tune a recipe. Such an agent can search the framework's full argument space, while deterministic checks outside the agent decide which measurements count and which recipes are correct.

We present ATTUNE, an agentic tuner that starts from any working recipe, including an expert's, and improves its training throughput, unattended, for the user's workload and stack within a GPU-hour budget the user sets. One such tuning session is a *campaign*. A single coding agent runs the whole loop, guided by a tuning *skill* that holds its search rules, helper scripts and a catalog of levers distilled from past campaigns. The agent diagnoses each run, reads a flag's implementation before using it and changes one lever at a time (Section 2.2).

ATTUNE ships a recipe only if it passes a correctness gate with two checks: a configuration check refuses any argument that changes the objective or hits a known stack defect, and a loss check compares the recipe's loss at its first training step with the baseline's. Every other change is judged by the loss check and listed for review. A harness runs the loop from one command to a shipped recipe. Each campaign's lessons feed back into the skill after review. ATTUNE tunes the production supervised fine-tuning (SFT) recipes for Amazon's post-training jobs on H100, H200 and B200 clusters, raising throughput over the recipes they replaced by 1.7× on average and up to 3.9× (Section 6.1).

We evaluate ATTUNE on two sets of public recipes. The first consists of four of NVIDIA's NeMo AutoModel benchmark recipes, run unchanged on NVIDIA's framework and the H100s they were written for, where our runs come close to NVIDIA's published throughput. The second consists of four models held out from ATTUNE's development, on H100 and B200: Llama-3.1-405B (Llama Team, AI @ Meta, 2024), Mixtral-8x22B (Jiang et al., 2024a; Mistral AI, 2024), GLM-4.5 (GLM-4.5 Team, 2025) and MiniMax-M2 (MiniMax, 2025), each at its public recipe's workload on real text. Each of these campaigns starts from the strongest public recipe for its model: vendor throughput benchmarks for Llama-3.1-405B and Mixtral-8x22B, a library recipe written for H100 for GLM-4.5, and a recipe ported to fewer GPUs for MiniMax-M2. We ask how far ATTUNE improves that recipe and how it compares with OptiCo, Bayesian optimization and random search when all get the same GPU-hour budget, isolated environment and correctness gate. To find which design choices matter, we ablate three of them on two more held-out models.

Our contributions:

- **ATTUNE**, a single-agent throughput tuner that runs unattended from a working recipe to a shipped one (skill, harness and correctness tools, open source at `github.com/amazon-science/attune`). It can change any framework argument that its correctness gate does not mark as objective-changing.
- **A correctness gate for throughput tuning**, enforced outside the tuner. On runs with a known answer, it passes all 460 legitimate runs and refuses every objective-changing argument and known stack defect before launch. It also catches code faults that change the first training step. Faults that act only through the optimizer update, and a shifted loss mask, pass it. Existing tuners check only that a configuration runs or compiles, not that it trains the same objective.
- **An evaluation on eight public recipes.** On four of NVIDIA's AutoModel benchmark recipes, ATTUNE's recipes exceed NVIDIA's published throughput by 6–17%. On four held-out models, under one protocol with scoring, correctness checks and a single decision rule run by the control plane for all tuners (Section 3.1), ATTUNE improves the strongest public recipe by up to 2.6×, most where the recipe was written for another setting. Under a common GPU-hour budget it beats all other tuners on GLM-4.5 and MiniMax-M2. On GLM-4.5 its gain is more than four times that of the best alternative, using less than half the GPU-hours of any. Ablations on two more held-out models show that the same agent without ATTUNE's skill gains less and fails more launches, and that a fixed knob schema falls behind when a model's largest lever lies outside it.
- **OpenRecipes**, an open leaderboard of more than 60 tuned and verified recipes for more than 20 models, each with its throughput, its cost per million tokens and a passed convergence check (Section 4).

2 DESIGN

ATTUNE separates recipe tuning into an agent-driven search and deterministic checks. A coding agent, guided by a tuning skill, runs the search: it diagnoses each run, forms a hypothesis, changes the recipe and launches the job. Deterministic code decides which measurements count, which candidate leads and which recipe ships, and a harness keeps the loop running unattended (Figure 1).

Each setting of a recipe that the search may change is a *lever*, usually a framework argument (flag). A *campaign* tunes one model on one cluster at a fixed GPU count. It starts from a working *baseline* recipe and runs under a GPU-hour budget. Each *candidate* recipe runs a short benchmark, and the best recipe so far is the *leader*, the one each new candidate must beat. The campaign ends when the budget is spent, or earlier if the agent judges that the search has converged and records its evidence: the leader, the levers it

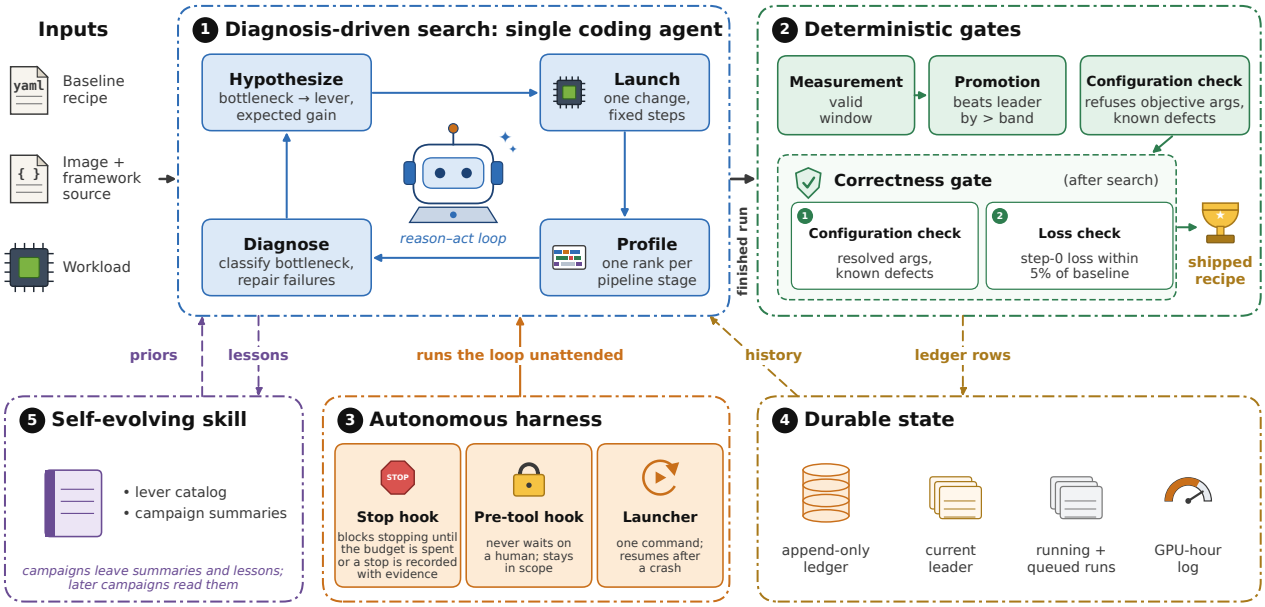


Figure 1. ATTUNE's architecture. (1) A single coding agent runs the search as a reason–act loop: it states a hypothesis, launches one change, profiles the run and diagnoses the bottleneck. (2) Deterministic gates outside the agent refuse objective-changing flags, decide whether a measurement is valid and whether a candidate beats the leader, and, after the search, check the leader before it ships. (3) The harness keeps the loop running unattended, (4) durable state records every run so a campaign can resume, and (5) the skill passes the lever catalog and the summaries and lessons of past campaigns to later campaigns.

tested and its last diagnosis.

2.1 Measurement

Workload. A campaign holds its workload identical across candidates: the model, dataset, sequence length, global batch, precision and GPU count, and every run reads the same rows in the same order. Before tuning, ATTUNE measures the dataset's token-length distribution. Every micro-batch must have room for the longest row, so the longest rows decide which layouts fit. The agent reads these limits before its first hypothesis.

Scoring. Each run lasts a fixed number of optimizer steps, set by the operator. The first steps absorb startup and compilation and are discarded, and the rest form the measured window W. We score a recipe x on the set G of its GPUs by its aggregate throughput over the window, where tokens_i counts the tokens all GPUs process in step i:

$$s(x) = \frac{\sum_{i \in W} \text{tokens}_i}{|G| \sum_{i \in W} \text{stepTime}_i}$$

and the search maximizes $s(x)$. Summing before dividing keeps short steps from weighing more than long ones.

Baseline and band. Throughput varies from run to run, so a gain must exceed this noise to count. We run the baseline at least five times on different node allocations. The mean of these runs is the denominator for gains during search, and their relative standard deviation σ sets the noise band, $\max(3\%, 3\sigma)$. A candidate replaces the leader only when it beats it by more than the band.

2.2 Search

2.2.1 Search loop

Agent loop. The agent runs a reason–act loop with ordinary tools: file search and reading, including the framework source in the training image, a shell and the cluster API. A tuning skill supplies the loop, the measurement rules, a lever catalog and helper scripts. The skill instructs the agent to prioritize candidates by expected gain and confidence. To estimate both, the skill points it to four sources: its diagnosis of what limits the leader, the lever catalog, the ledger of the campaign's past runs and the flags in the framework source. The catalog records each lever's mechanism, when it helps, its direction and its rough size.

Search discipline. The skill gives the agent six rules:

1. State a hypothesis before launching each candidate: the bottleneck it should shrink, the mechanism and the expected gain.
2. Change one lever, or one coupled set of parallelism degrees, per candidate, so each result can be attributed.
3. Before using a flag, read its implementation in the running image, including what it asserts and which other flags it silently overrides, and record the code location in the ledger.
4. Rule out a lever only after measuring two of its values on the same recipe, or after reaching the lever's feasible limit. Conclude that topology is exhausted only after probing each of TP, PP, EP, CP and DP.
5. When trying a new layout, start without the leader's memory savings (e.g., full recomputation) unless a memory estimate shows the new layout needs them. If the new layout runs out of memory, add them back and measure again before ruling it out.
6. Once the layout stops changing, re-test the flags that won on earlier layouts, since a flag's gain often does not carry over to another layout.

2.2.2 *Diagnosis and profiling*

The agent diagnoses each run from its metrics and training log. The main signals are step time, per-stage memory and step-time variance within the window. When the bottleneck is still unclear, it profiles a run. A profiled run traces one rank per pipeline stage. A deterministic trace analyzer splits each traced stage's step into compute, communication, host-side and idle time, so the agent sees which stage is slowest and what it waits on.

2.2.3 *Control plane*

The control plane is a set of deterministic scripts that launch the agent's candidates and record their results. To launch a candidate, a run generator copies the baseline job and changes only the flags and allowlisted environment variables under test, so the rest of the job stays identical. An accounting script charges every job the GPU-hours from its admission by the scheduler to its release, including failed runs and correctness checks, and admits a new launch only if its expected cost fits the remaining budget. The control plane also refuses to relaunch a recipe, so after a failure the agent must diagnose it and change the recipe.

2.2.4 *Correctness gate*

A tuned recipe must train the same objective as its baseline. The gate tests this with two checks. Before launch, a *configuration check* compares the candidate's resolved settings with the baseline's. After the run, a *loss check* compares their losses at the first training step. The gate is designed never to reject a recipe that keeps the objective, since a false reject throws away a correct speedup. As a result, a fault too subtle to show in a short run can pass.

Configuration check. The control plane compares the candidate's resolved arguments and environment with the baseline's. It uses resolved values, not the flags as written, because one flag can silently change others (Section 1). It looks up each change in a registry that no tuner can edit and sorts it into one of three classes: known invalid, known valid and unknown. A *known invalid* change alters what is trained: the data, the batch, the optimizer or its schedule, a loss term, routing or the precision format; known stack defects of the model and image also count (Table B1). A *known valid* change alters only how the same computation runs: a placement change, a packing change (how rows fill microbatches) or a same-math swap that passes its test. A *same-math swap* computes the same function by another path, such as a fused kernel; its *component test* compares it with an FP32 reference on identical inputs (Appendix B), and a swap that fails counts as a defect. An *unknown* change is any other change. The configuration check refuses known invalid changes, passes known valid ones, and lets unknown ones through to the loss check, listing them in the gate report for review.

Loss check. The candidate's loss at the first training step must lie within 5% of the baseline's, and all logged losses must be finite. The first step (step 0) applies the same loaded weights to the same first batch before any update, so a legitimate change can move this loss only through floating-point evaluation order, far less than 5%. Faults that load or shard weights wrongly move it by multiples (Section 3.5).

Shipping. The control plane ships the fastest candidate that beats the baseline by more than the band and passes the gate, and otherwise the baseline. Megatron-Bridge's convergence check (NVIDIA, 2026b) on the loss curves is reported for review only, because on a model with noisy early loss it fails a fifth of identical-recipe pairs (Appendix B).

2.3 Harness

Autonomy. ATTUNE builds on the *persistent goal* feature of coding-agent harnesses such as Claude Code (Anthropic, 2026), which keeps one objective active across many agent turns. A stop hook blocks the agent from ending its turn until the campaign has spent its budget or recorded a stop with evidence. A pre-tool hook turns any question to the user into an instruction to take the default and record the assumption, and blocks unrecognized destructive shell commands. A launcher starts the campaign with one command and resumes the session after a crash.

Durable state. An append-only ledger is the campaign's source of truth. Each row records one run: its recipe, each lever's registry class, the diagnosis, the measurement, the verdict against the leader and the GPU-hours. A fresh session rebuilds the campaign from the ledger and continues.

Self-evolution. At the end of a campaign, the agent writes a short summary for later campaigns to read. During the campaign, when a run establishes a generalizable fact about a lever, the agent writes it as a lesson and records the evidence and workload behind it. These lessons enter the lever catalog after manual review.

3 EVALUATION

Our evaluation answers four questions:

- How far does ATTUNE improve the strongest public recipe, reproduced on NVIDIA's AutoModel or ported to our stack (Section 3.2)?
- How does ATTUNE compare with OptiCo, Bayesian optimization and random search under the same GPU-hour budget (Section 3.3)?
- How much does each of ATTUNE's design choices contribute to its gains (Section 3.4)?
- Does ATTUNE's correctness gate pass every legitimate change, and which faults does it catch (Section 3.5)?

3.1 Setup

Models. We use two sets of public recipes (Table A3). The *reproduced* set holds four of NVIDIA's NeMo AutoModel benchmark recipes, run unchanged in NVIDIA's container on the H100s they were written for (NVIDIA, 2026d): Qwen3-30B-A3B (Qwen Team, 2025), GPT-OSS-20B (OpenAI, 2025), Nemotron-3-Nano (NVIDIA, 2025) and Llama-3.1-70B (Llama Team, AI @ Meta, 2024). Our runs of three of them come within 2% of the published throughput. These models are not held out: Qwen3-30B-A3B and GPT-OSS-20B are development models, and Nemotron-3-Nano is an ablation model, all tuned before on slime or miles (Appendix G). The *ported* set holds the four held-out models, run on slime (Zhu et al., 2025b) and Megatron-LM: a dense 405B model (Llama-3.1-405B) and three MoE models from 141B to 355B parameters (Mixtral-8x22B, MiniMax-M2, GLM-4.5). Two of these recipes are vendor throughput benchmarks for the hardware we run them on (Llama-3.1-405B, Mixtral-8x22B); GLM-4.5's is a library recipe written for H100 that gives a layout but no throughput.

Workload. Each campaign keeps its recipe's sequence length, global batch, precision and GPU count, so a gain is measured against the recipe at the shape it was written for; the one exception is MiniMax-M2, whose 128-GPU recipe we port to 64 GPUs. The reproduced set keeps NVIDIA's benchmark workload: mock tokens with balanced routing, from random initialization except on Llama-3.1-70B. Because the loss check is only a sanity check on that workload, we also train each shipped recipe on real text from the released weights, alongside the public recipe (Appendix D). The ported set trains on real text from the released weights.

Baseline and tuners. Every tuner starts from the same baseline: the public recipe on that workload. In the ported set the baseline carries the changes of Appendix A, and Table D1 gives its throughput and noise band. The ATTUNE arms run a frozen skill with no summaries of past campaigns (Appendix G). The other tuners keep their own algorithms (Section 3.3); OptiCo keeps its analyzer, prompts and schema (Appendix E). All agents use Claude Opus 4.8 with web tools disabled. Each campaign runs in its own sandbox and reaches the cluster only through a broker (Appendix C). The reproduced set uses a lighter protocol. Each model gets one campaign, with its own budget (Table 1). The agent is Claude Opus 5.5 with web tools enabled. It runs the released skill with an AutoModel adapter, without the sandbox or broker. Runs are scored over the benchmark's own window, with no final re-measurement.

Budget, scoring and comparisons. In the ported set every tuner gets the same GPU-hour budget per model: 30 times the cost of one baseline benchmark run (Table D1). All tuners share the correctness gate and the shipping rule (Section 2.2.4). Each benchmark run lasts 14 optimizer steps, scored over steps 4–13. Reported throughputs come from a final interleaved re-measurement of the baseline and every shipped recipe (Appendix D). Comparisons use the *mean rule*: arm A beats arm B when mean(A) exceeds mean(B) by more than the model's noise band. Each comparison also reports an exact permutation p-value (Appendix D).

3.2 ATTUNE against the strongest public recipes

Table 1 compares the throughput of each public recipe and its ATTUNE-tuned recipe, at the public recipe's own workload and GPU count. ATTUNE improves throughput in both the reproduced and the ported set.

Reproduced recipes. We first reproduce NVIDIA's four AutoModel benchmark recipes on our cluster. Three come within 2% of NVIDIA's published throughput, and Llama-3.1-70B within 8%. Starting from these reproductions, ATTUNE raises throughput by 7–26%, which puts its recipes 6–17% above NVIDIA's published figures. The largest gain is on Llama-3.1-70B, where ATTUNE drops context parallelism and switches attention to FlashAttention-3; the step becomes 98.5% compute. On Qwen3-30B-A3B and Nemotron-3-Nano the gain comes from the expert dis-

patcher's kernel and SM budget. On GPT-OSS-20B it also comes from FSDP scheduling and faster attention and expert kernels. On real text from the released weights, the changes in every shipped recipe train within run-to-run noise of the public recipe (Appendix D).

Ported recipes. On the models whose recipes we ported to our stack, ATTUNE raises throughput by 164.2% on MiniMax-M2, 33.9% on GLM-4.5 and 6.8% on Llama-3.1-405B. On Mixtral-8x22B its best candidate stays inside the noise band, so it keeps the public recipe. The gain is largest where the recipe was written for another setting. MiniMax-M2's recipe was written for reinforcement learning on 128 GPUs and keeps the optimizer state in CPU memory. GLM-4.5's recipe was written for H100 and leaves room to tune on B200. The Llama-3.1-405B and Mixtral-8x22B recipes are vendor benchmarks written for the hardware they run on, so they leave less. On MiniMax-M2, all three campaigns turn the CPU optimizer offload off and raise pipeline parallelism. On GLM-4.5, ATTUNE found the step bound by communication and kernel launches. All three campaigns moved the expert all-to-all inside a node, and the fastest also dropped tensor parallelism to one (Appendix F).

3.3 Tuner comparison

We compare ATTUNE with an LLM-agent tuner and three black-box searches on GLM-4.5 and MiniMax-M2. Every tuner gets the same GPU-hour budget, correctness gate and shipping rule. Each tuner runs three campaigns per model, because a campaign is stochastic.

OptiCo. We run the open-source OptiCo (Chen et al., 2026) with the same agent model as ATTUNE, through an adapter for our stack and our loop driver (Appendix E). We keep its published search space, a fixed schema of 11 knobs. The schema leaves out expert-tensor and context parallelism, virtual stages, the dispatcher and optimizer offload, so these stay at the baseline's values.

Black-box search. We run three black-box searches over each model's frozen space of objective-preserving knobs (Appendix E): uniform random search, Bayesian optimization (TPE; Bergstra et al., 2011; Akiba et al., 2019) and the same with an analytical memory pre-filter that rejects configurations predicted to run out of GPU memory.

ATTUNE ships faster recipes than the other tuners. On GLM-4.5 its mean gain is more than four times that of the best alternative, random search (Table 2). On MiniMax-M2 it is about three times that of Bayesian optimization with the pre-filter. Each of ATTUNE's three GLM-4.5 campaigns gains at least 22%, while eight of the other tuners' 12 campaigns ship the public recipe. Each of its MiniMax-M2 campaigns is faster than the other tuners' fastest.

ATTUNE uses the least search compute. It needs the fewest GPU-hours and launches on both models, and the least wall-clock time on GLM-4.5 (Table 2, Figure 2). Every GLM-4.5 ATTUNE campaign finds a recipe more than 20% faster than the public recipe within the first 30% of the budget. The campaigns then stop well before the budget runs out, so on average ATTUNE uses about half the GPU-hours of any other tuner. On both models its tuning pays for itself within a day of training on its recipe, at least four times sooner than any other tuner (Appendix D).

The other tuners miss winning layouts within their reach. ATTUNE's fastest GLM-4.5 layout, without its virtual stages, lies inside OptiCo's schema and measures within 2% of ATTUNE's recipe, yet OptiCo did not reach it. OptiCo tried tensor parallelism one only at shallower pipelines, with expert parallelism still crossing nodes, and none of its runs beat the public recipe. The black-box searches have all of GLM-4.5's winning levers in their space but no diagnosis to guide them. Most of their draws ran out of memory or were pruned as slower than the leader, and with the pre-filter most of the remaining draws were still pruned. ATTUNE instead names the bottleneck, here the cross-node expert all-to-all and the tensor-parallel communication. It changes the coupled degrees together and reaches a winner in eight launches or fewer. On MiniMax-M2, eight of the nine black-box campaigns drop the CPU optimizer offload, a change worth about a third of ATTUNE's mean gain (Table D3). Every ATTUNE campaign adds a deeper pipeline of four or eight stages to that change, with tensor parallelism one, more rows per microbatch or both.

3.4 Ablations

To better understand what each of ATTUNE's design choices contributes, we remove or replace one at a time and compare the result with full ATTUNE (Table 3). The *bare agent* is the same agent and harness without the skill. The *fixed-knob* arm may change only the 11 knobs of OptiCo's schema (Section 3.3). The *no-diagnosis* arm sees scores and run state but no logs or profiles. The ablations use two more held-out models, GLM-4.7-Flash (Zhipu AI, 2026) and Nemotron-3-Nano-30B-A3B (NVIDIA, 2025), trained with SFT on miles (RadixArk, 2026) at 8×H200, starting from the recipe miles publishes for each model. Every arm runs five campaigns per model under the same GPU-hour budget, and each campaign sees only its own records (Appendix D). Full ATTUNE has the highest mean gain on both models, and its weakest campaign beats every other arm's weakest.

Table 1. ATTUNE's gain over the strongest public recipe, at that recipe's workload shape and GPU count. MiniMax-M2's recipe, written for 128 GPUs, runs on 64. Baseline is the public recipe on our cluster: for the reproduced set unchanged in NVIDIA's container, for the ported set on slime and Megatron-LM with the changes of Appendix A (Table A3). In the reproduced set each model has one campaign; the gain over published compares ATTUNE with NVIDIA's published figure (Table A3), measured on other machines and software versions. In the ported set, values over three campaigns are means, judged by the mean rule (Section 3.1), and no gain over the published figures is given because they come from other stacks. MFU (Appendix D) is given as baseline → ATTUNE, and GPU-h is the GPU-hours spent per campaign, with its budget in parentheses. Tables D2 and D3 give every campaign.

		Throughput (tok/s/GPU)		Gain over			
Model	GPUs	Baseline	ATTUNE	Baseline	Published	MFU (%)	GPU-h (budget)
Reproduced on NeMo AutoModel							
Qwen3-30B-A3B	8×H100	11,963.1	13,151.1	+9.9%	+9.2%	27.9 → 30.7	37.2 (70)
GPT-OSS-20B	8×H100	12,970.3	15,222.5	+17.4%	+16.6%	30.1 → 35.3	18.1 (65)
Nemotron-3-Nano	8×H100	16,538.1	17,726.7	+7.2%	+5.6%	33.4 → 35.8	23.5 (65)
Llama-3.1-70B	64×H100	801.2	1,012.5	+26.4%	+16.8%	36.4 → 46.0	133.4 (500)
Ported to slime and Megatron-LM							
GLM-4.5	128×B200	723.6	969.0	+33.9%	–	7.2 → 9.7	669.1 (1,880)
MiniMax-M2	64×B200	895.2	2,365.1	+164.2%	–	3.2 → 8.5	203.5 (530)
Llama-3.1-405B	128×B200	394.9	421.9	+6.8%	–	22.2 → 23.7	393.9 (1,850)
Mixtral-8x22B	128×H100	1,072.4	1,072.4	0.0%	–	26.3	173.4 (740)

Table 2. Tuner comparison on GLM-4.5 / 128×B200 and MiniMax-M2 / 64×B200, means over three campaigns. ATTUNE's rows are the campaigns of Table 1. Gain is over the public recipe. Failed launches gives the share of launches that ran out of memory, hung or crashed, with their GPU-hours in parentheses; Appendix D defines the columns. Bold marks each model's best value per column. On both models ATTUNE beats every other tuner under the mean rule, with exact permutation p = 0.05 against each, the lowest that test allows (0.10 against random search on GLM-4.5), and block-stratified p = 0.125, also that test's lowest. BO: Bayesian optimization; TPE: tree-structured Parzen estimator.

Model	Tuner	Gain	GPU-h used	Launches	Failed launches (GPU-h)	Wall-clock
GLM-4.5	ATTUNE	**+33.9%**	**669**	**8.3**	24% (60)	**6.4 h**
	OptiCo	0.0%	1,407	22.7	19% (78)	15.7 h
	BO (TPE)	+1.8%	1,675	19.7	39% (186)	9.6 h
	BO + pre-filter	+3.9%	1,695	13.7	**5% (58)**	9.5 h
	Random search	+7.8%	1,688	22.7	51% (278)	10.2 h
MiniMax-M2	ATTUNE	**+164.2%**	**203**	**13.3**	23% (25)	3.1 h
	OptiCo	+22.4%	386	26.0	24% (32)	6.9 h
	BO (TPE)	+46.6%	492	35.0	53% (136)	**2.7 h**
	BO + pre-filter	+55.5%	489	22.7	**4% (23)**	2.9 h
	Random search	+32.7%	487	38.0	67% (202)	**2.7 h**

The skill. Without the skill, the agent's mean gain is 24% lower on GLM-4.7-Flash and 13% lower on Nemotron-3-Nano. It also fails far more of its launches and needs up to 2.3 times the GPU-hours to reach a 20% gain (Table 3).

The open action space. Restricted to OptiCo's knobs, the agent falls far behind on GLM-4.7-Flash and ties on Nemotron-3-Nano. The difference is where each model's largest win lies: on GLM-4.7-Flash it is turning CPU optimizer offload off, which the schema does not list, while Nemotron-3-Nano's largest wins lie inside the schema.

Diagnosis. Without logs and profiles, the agent's mean gain is lower than full ATTUNE's on both models, +73.4% against +88.0% on GLM-4.7-Flash and +47.5% against +49.4% on Nemotron-3-Nano, but with five campaigns per arm the differences are not significant (p = 0.24 and 0.14; Table 3).

3.5 The correctness gate

To check that the gate passes legitimate changes and catches faults, we test it on two sets of runs. The first are runs whose correct verdict we know in advance (Table 4): repeats of public recipes, legitimate recipes whose step-0 numerics differ from the baseline's, injected faults, and the ablation runs of Section 3.4. The second are all candidates of the evaluation and ablation campaigns.

Table 3. Ablations on GLM-4.7-Flash and Nemotron-3-Nano-30B-A3B / 8×H200, five campaigns per arm and model. Gain is the mean over each campaign's own baseline run, with the range in parentheses; p is a one-sided exact permutation test of full ATTUNE against the arm (at least 0.004). Failed is the share of launches that crashed or did not complete; GPU-h to +20% is the median GPU-hours until a run passing the gate first gains 20%, over the campaigns that get there (count in parentheses). Bold marks the best value per column.

Arm	GLM-4.7-Flash gain, %	p	Failed	GPU-h to +20%	Nemotron-3-Nano gain, %	p	Failed	GPU-h to +20%
Full ATTUNE	**+88.0** (81.7–96.6)	–	17%	**3.9** (5/5)	**+49.4** (47.1–51.4)	–	**32%**	5.2 (5/5)
Bare agent	+66.9 (54.7–94.5)	0.016	50%	9.0 (5/5)	+42.9 (35.9–48.8)	0.008	50%	7.2 (5/5)
Fixed knobs	+18.1 (15.6–21.4)	0.004	**13%**	19.8 (2/5)	+48.6 (41.3–51.3)	0.42	33%	**3.1** (5/5)
No diagnosis	+73.4 (21.3–96.5)	0.24	19%	4.9 (5/5)	+47.5 (41.9–50.2)	0.14	41%	**3.1** (5/5)

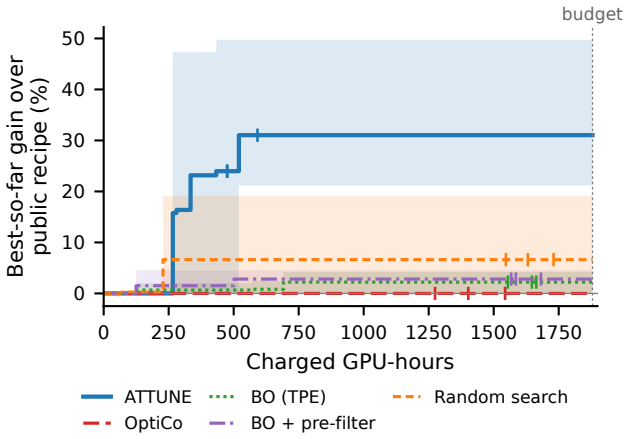


Figure 2. Best-so-far search-time gain over the public recipe's calibration mean (728.2 tokens/s/GPU) against cumulative charged GPU-hours on GLM-4.5 / 128×B200: the mean over each arm's three campaigns (line) and their range (shading). Ticks mark each campaign's last search run; the curve then runs flat to the 1,880 GPU-hour budget (dotted line).

Known-answer runs. Across the 460 legitimate and 43 faulty runs of Table 4, the gate passes every legitimate run and rejects every configuration fault. Of the six code faults, the loss check catches the two that change the forward pass, which raise the step-0 loss by more than 150%. It passes the other four. Three act only through the update, such as a skipped gradient reduction, which step 0 cannot see, and a loss mask shifted by one token moves the step-0 loss by 2.0%, about as much as the largest legitimate change. ATTUNE changes only configuration, not code, and where such a fault has a configuration form, such as a changed auxiliary-loss coefficient, the configuration check refuses it (Table B2). Two real stack defects that a configuration change can trigger, EP 1 with ETP of 4 or more and PP 32, move the step-0 loss by 0.02% or less, so only the configuration check's list of known defects can stop them.

In the campaigns. In the ported-set campaigns of Sections 3.2–3.3 and the GLM-4.5 ablation arms of Appendix D, every candidate that reached the gate passes, with step-0 shifts of at most 0.03%, far inside the 5% tolerance, so the loss check never came close to rejecting a legitimate candidate. The configuration check refused 11 launches that changed tensor-parallel communication overlap, a legitimate swap that had no component test at the time. Refusing untested swaps has this cost, and the released gate now passes this swap to the loss check. In the ablation campaigns of Section 3.4, the configuration check refused 20 candidates with known defects: 19 uneven pipeline splits on Nemotron-3-Nano and one fused router on GLM-4.7-Flash. In one campaign the fastest run was such a split, which loads another layer's weights into the Mamba layers (Table B1); the gate correctly refused it and shipped the next fastest candidate. To check that a recipe that passes the gate also holds over a longer run, we trained one shipped recipe for 1,000 steps; its loss stayed within run-to-run noise of its public recipe's loss (Appendix B).

4 An Open Leaderboard of Tuned Recipes

Few open training recipes are tuned, and fewer report their throughput. The published figures cover a handful of model and accelerator pairs, each with its own workload and metric, so a practitioner cannot tell which recipe is fastest for a given model and accelerator. We release OpenRecipes (`https://openrecipes.ai`), an open leaderboard of training recipes measured under one protocol: 8,192-token SFT rows at global batch 128 in BF16, scored over steps 5–19 of a 20-step run (Appendix A). Each entry lists its tokens/s/GPU, its cost per million tokens, and the exact recipe and framework versions. An entry may change only what the configuration check marks as known valid: placement, packing and tested same-math swaps (Section 2.2.4). It must

Table 4. The correctness gate on runs with a known answer. Bold cells are correct decisions. A fault tested on several models counts once per model. Legitimate ablation runs change only placement, packing or same-math swaps with a passed component test. Table B2 details the first and third rows.

Runs with a known answer	Gate passes	Gate rejects
Legitimate: 132 runs on six models	**132**	0
Legitimate: 328 ablation runs on two models (Section 3.4)	**328**	0
Faulty: 24 runs of 14 faults	9 runs of four faults that move the first step's loss by 2% or less: one gradient bucket's reduction skipped (five models), load-balancing loss zeroed, loss mask shifted by one token, QK-norm gradients summed over tensor-parallel ranks	**15**: 6 objective-changing arguments and 4 configuration defects, refused before launch; 5 code faults that change the forward pass (labels offset by one token, attention sinks ignored), caught at step 0
Faulty: 19 ablation runs with an uneven pipeline split on Nemotron-3-Nano-30B-A3B	0	**19**, refused as a known stack defect; step-0 loss 215–768% above the baseline's

also pass Megatron-Bridge's convergence check against the recipe it was tuned from. OpenRecipes lists more than 60 recipes for more than 20 models on H100, H200 and B200. Its entries are seeded from ATTUNE campaigns, and the leaderboard is open to submissions. Teams can adopt a listed recipe instead of repeating the search. They can also use its cost to weigh in-house training against a paid service.

5 Related Work

Agentic LLM tuners. The systems closest to ATTUNE are OptiCo (Chen et al., 2026), PROMPTS (Ding et al., 2026b) and ASAP (Ding et al., 2025). OptiCo's LLM agents launch training jobs, read the results and propose the next configuration. PROMPTS and ASAP read the profile of an existing run and propose sharding configurations for an engineer to run. CCL-Search (Ding et al., 2026a) runs a similar loop with a single LLM over knobs the user lists, and Megatron-Bridge's agent skills (NVIDIA, 2026b) guide profiling-driven tuning in an interactive session. Other LLM agents tune GNN data prefetching (Sarkar et al., 2026), parallel-filesystem parameters (Egersdoerfer et al., 2025), GPU kernels (Damani et al., 2024; Varadharajan et al., 2026) and database knobs (Lao et al., 2024), evolve code against measured outcomes (Cheng et al., 2025; Novikov et al., 2025) and automate LLM fine-tuning (Ma et al., 2026).

ATTUNE shares OptiCo's run-and-measure loop and differs from these systems in four ways. First, it is a single agent under a skill, while OptiCo coordinates four agents and PROMPTS and ASAP use a coordinator–analyzer–proposal pipeline. A single agent can match multi-agent discussion (Wang et al., 2024), and committees add coordination failures (Cemri et al., 2025) and cost (Smit et al., 2024); a committee arm we ran was cheaper than the single agent but less consistent (Appendix D). Second, it can change any framework argument its gate does not mark as objective-changing, while PROMPTS and ASAP tune the parallelism mapping and OptiCo an 11-knob schema. Third, it checks that a change preserves the training objective: OptiCo checks that a configuration runs, PROMPTS that it compiles, and CCL-Search and the Megatron-Bridge skills check neither. Fourth, it ships without a human and carries lever rules across campaigns, while PROMPTS returns three recommendations for human review and OptiCo's memory lasts one optimization. We run OptiCo as a baseline (Section 3.3); PROMPTS and ASAP release no code and target TPUs.

Planners and autoconfigurators. Planners search the parallelization space with a cost model or simulator. vTrain (Bang et al., 2024), SimAI (Wang et al., 2025) and FlexFlow (Jia et al., 2019) estimate a plan's step time without running it; Alpa (Zheng et al., 2022), Galvatron (Miao et al., 2022), Mist (Zhu et al., 2025a), Metis (Um et al., 2024), nnScaler (Lin et al., 2024) and Yuan & Zhao (2026) synthesize plans under memory constraints; Sailor (Strati et al., 2025), CubicML (Wen et al., 2024), EasyScale (Li et al., 2023), Rubick (Zhang et al., 2025) and AutoScout (Shong et al., 2026) extend planning to heterogeneous, elastic and shared clusters and to runtime parameters; and Aceso (Liu et al., 2024) iteratively relieves the profiled bottleneck. DeepSpeed's autotuner (DeepSpeed Team, 2021) and NeMo's AutoConfigurator (NVIDIA, 2026c) sweep a bounded set of documented knobs; AutoConfigurator grids only the parallel degrees and the microbatch size. Both search a space fixed in advance, mostly placement. ATTUNE measures candidates on the target cluster. On GLM-4.5 and MiniMax-M2 its recipes combine placement with a dispatcher swap and changes to optimizer offload (Ren et al., 2021) and communication overlap (Tables D2 and D3), which lie outside a placement planner's space.

Table 5. Throughput gain of ATTUNE's recipes in Amazon's post-training jobs over the recipes they replaced, on 8,192-token rows at global batch 128; the geometric mean is 1.7×. Table D5 gives the GPU counts, the replaced recipes and the cost per million tokens.

Model	H100	H200	B200
Qwen3.5-122B	2.49×	1.32×	1.26×
Qwen3.5-397B	1.25×	1.06×	3.91×
GLM-5.2	1.07×	2.54×	2.21×

6 DISCUSSION

6.1 Deployment

ATTUNE tunes the SFT recipes that Amazon's post-training jobs run for three models on H100, H200 and B200 clusters (Table 5). Each tuned recipe replaced a public or in-house recipe after a loss-parity check against it. The tuned recipes raise throughput by 1.7× on average and up to 3.9×. Of the production jobs on these models, 65% use them. The largest gains come from changing how a recipe trades memory for speed. On B200, for example, the Qwen3.5-397B recipe drops CPU offload and runs on half the GPUs.

6.2 Lessons learned

The catalog needs curation before reuse. Our past campaigns wrote their lessons into the catalog as general rules. When we later checked the catalog against the campaigns' own records, about a fifth of the entries were wrong, and several held only on the workload they were measured on (Table G1). On GLM-4.5, one campaign followed a wrong entry when it first turned tensor parallelism off, and it never reached the winning layout. Many correct entries were also redundant, since a current model asked without the skill already knew most entries at least in part. For about a quarter of them, however, its answer contradicted the measured result. For example, it expects fusion and overlap flags to add up to a large gain, while in our campaigns they tied or lost. A catalog pays only where it corrects the model or states facts about the stack that the model cannot know. Each campaign therefore records its lessons separately from the catalog, each with the runs that support it and the workload it was measured on. Lessons enter the catalog only after review, and we keep only the entries the current model misses.

The skill's search rules need enforcement. The skill tells the agent to retry a layout that runs out of memory, with the memory savings added back, before ruling it out. On GLM-4.5, the campaigns that followed this rule shipped the best recipes. The campaigns that skipped the retry missed the largest gains (Appendix D). The harness already enforces the correctness gate and the budget outside the agent. Enforcing the costliest search rules the same way is the next step.

6.3 Limitations

Priors. The lever catalog was distilled from past campaigns. One held-out model, GLM-4.5, belongs to the same family as the development model GLM-5.2. Three models of the reproduced set were tuned before: Qwen3-30B-A3B and GPT-OSS-20B on slime, and Nemotron-3-Nano on miles. Knowledge carried over from these campaigns could account for part of ATTUNE's gains on those models. On GLM-4.5, ATTUNE without the catalog matches full ATTUNE within the noise band (Table D4), which suggests this contribution is small.

Gate coverage. The gate decides from a candidate's configuration and its first training step. It therefore misses faults that act only through the optimizer update (Section 3.5). Likewise, a flag the configuration check does not know can trigger a framework bug that leaves the first-step loss unchanged. The gate report lists the unknown flags in each shipped recipe so that a reviewer can check them. A recipe should still be validated with a full training run before production use. On mock-token benchmarks from random initialization, the first-step loss carries little signal, so we validate those recipes on real text (Appendix D).

Scope. ATTUNE tunes the configuration that the framework and the training image expose. It leaves framework code, kernels and the image unchanged. It covers the training step. The rollout and generation phases of RL training lie outside its scope. The recipe it returns is the best its search found for that cluster and stack, with no guarantee of optimality or of transfer to another setting.

7 CONCLUSION

ATTUNE is an agentic throughput tuner that tunes multi-node LLM training recipes unattended, through a single coding agent guided by a tuning skill and a correctness gate enforced outside the agent. Experiments show that ATTUNE exceeds NVIDIA's published throughput on four of NVIDIA's own benchmark recipes by 6–17% and improves held-out public recipes by up to 2.6×. It also outperforms Bayesian optimization, random search and OptiCo. In production, its recipes raise the throughput of Amazon's post-training jobs by 1.7× on average. We release ATTUNE and OpenRecipes, an open leaderboard of more than 60 tuned and verified recipes, so that teams can start from a fast recipe that trains the same model instead of repeating the search.

REFERENCES

Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. Optuna: A Next-Generation Hyperparameter Optimization Framework. In *Proc. KDD '19*, pp. 2623–2631, 2019.

Anthropic. Claude Code (v2.1.286) and Claude Opus 4.8. Claude Code documentation: overview and persistent goals. Accessed 2026-10-15, 2026. URL https://code.claude.com/docs/en/overview. Also https://code.claude.com/docs/en/goal.

Bang, J., Choi, Y., Kim, M., Kim, Y., and Rhu, M. vTrain: A Simulation Framework for Evaluating Cost-Effective and Compute-Optimal Large Language Model Training. In *Proc. MICRO '24*, pp. 153–167, 2024.

Bergstra, J., Bardenet, R., Bengio, Y., and Kégl, B. Algorithms for Hyper-Parameter Optimization. In *Proc. NIPS '11*, pp. 2546–2554, 2011.

Cemri, M., Pan, M. Z., Yang, S., Agrawal, L. A., Chopra, B., Tiwari, R., Keutzer, K., Parameswaran, A., Klein, D., Ramchandran, K., Zaharia, M., Gonzalez, J. E., and Stoica, I. Why Do Multi-Agent LLM Systems Fail? In *NeurIPS 2025 Datasets and Benchmarks Track*, 2025. arXiv:2503.13657.

Chen, S., Zhe, T., Zhang, W., Yang, F., Wang, Y., Li, T., and Liu, Y. OptiCo: Adaptive Distributed Training Optimization via Collaborative Agent Reasoning. In *Proc. ACL '26*, pp. 27847–27867, 2026.

Cheng, A., Liu, S., Pan, M., Li, Z., Wang, B., Krentsel, A., Xia, T., Cemri, M., Park, J., Yang, S., Chen, J., Agrawal, L., Desai, A., Xing, J., Sen, K., Zaharia, M., and Stoica, I. Barbarians at the Gate: How AI is Upending Systems Research. *arXiv preprint arXiv:2510.06189*, 2025.

Damani, S., Hari, S. K. S., Stephenson, M., and Kozyrakis, C. WarpDrive: An Agentic Workflow for Ninja GPU Transformations. In *NeurIPS 2024 Workshop on Machine Learning for Systems*, 2024.

DeepSpeed Team. DeepSpeed Autotuning, 2021. URL https://www.deepspeed.ai/tutorials/autotuning/.

Ding, E., Oh, B., Kataria, B., Guo, K., Gvero, J., Kumar, A. V., Devraj, A., Bowen, L., Sonwane, A., Manzoor, E., and Singh, R. CCL-Bench 1.0: A Trace-Based Benchmark for LLM Infrastructure. *arXiv preprint arXiv:2605.06544*, 2026a.

Ding, Y., Chen, X., Zhang, X., and Zhou, Z. ASAP: An Agentic Solution to Auto-optimize Performance of Large-Scale LLM Training. In *NeurIPS 2025 Workshop on Machine Learning for Systems*, 2025. arXiv:2511.03844.

Ding, Y., Han, R., Zhang, X., and Chen, X. PROMPTS: PeRformance Optimization via Multi-Agent Planning for LLM Training and Serving. In *Proc. MLSys '26 (Industry Track)*, pp. 1010–1039, 2026b.

Egersdoerfer, C., Carns, P., Snyder, S., Ross, R., and Dai, D. STELLAR: Storage Tuning Engine Leveraging LLM Autonomous Reasoning for High Performance Parallel File Systems. In *Proc. SC '25*, pp. 1252–1266, 2025.

GLM-4.5 Team. GLM-4.5: Agentic, Reasoning, and Coding (ARC) Foundation Models. *arXiv preprint arXiv:2508.06471*, 2025.

Jia, Z., Zaharia, M., and Aiken, A. Beyond Data and Model Parallelism for Deep Neural Networks. In *Proc. MLSys '19*, pp. 1–13, 2019.

Jiang, A. Q., Sablayrolles, A., Roux, A., Mensch, A., Savary, B., Bamford, C., Chaplot, D. S., de las Casas, D., Bou Hanna, E., Bressand, F., Lengyel, G., Bour, G., Lample, G., Renard Lavaud, L., Saulnier, L., Lachaux, M.-A., Stock, P., Subramanian, S., Yang, S., Antoniak, S., Le Scao, T., Gervet, T., Lavril, T., Wang, T., Lacroix, T., and El Sayed, W. Mixtral of Experts. *arXiv preprint arXiv:2401.04088*, 2024a.

Jiang, Z., Lin, H., Zhong, Y., Huang, Q., Chen, Y., Zhang, Z., Peng, Y., Li, X., Xie, C., Nong, S., Jia, Y., He, S., Chen, H., Bai, Z., Hou, Q., Yan, S., Zhou, D., Sheng, Y., Jiang, Z., Xu, H., Wei, H., Zhang, Z., Nie, P., Zou, L., Zhao, S., Xiang, L., Liu, Z., Li, Z., Jia, X., Ye, J., Jin, X., and Liu, X. MegaScale: Scaling Large Language Model Training to More Than 10,000 GPUs. In *Proc. NSDI '24*, pp. 745–760, 2024b.

Lambert, N., Morrison, J., Pyatkin, V., Huang, S., Ivison, H., Brahman, F., Miranda, L. J. V., Liu, A., Dziri, N., Lyu, X., Gu, Y., Malik, S., Graf, V., Hwang, J. D., Yang, J., Le Bras, R., Tafjord, O., Wilhelm, C., Soldaini, L., Smith, N. A., Wang, Y., Dasigi, P., and Hajishirzi, H. Tulu 3: Pushing Frontiers in Open Language Model Post-Training. In *Proc. COLM '25*, 2025.

Lao, J., Wang, Y., Li, Y., Wang, J., Zhang, Y., Cheng, Z., Chen, W., Tang, M., and Wang, J. GPTuner: A Manual-Reading Database Tuning System via GPT-Guided Bayesian Optimization. *Proc. VLDB Endowment*, 17(8):1939–1952, 2024.

Li, M., Xiao, W., Yang, H., Sun, B., Zhao, H., Ren, S., Luan, Z., Jia, X., Liu, Y., Li, Y., Lin, W., and Qian, D. EasyScale: Elastic Training with Consistent Accuracy and Improved Utilization on GPUs. In *Proc. SC '23*, pp. 1–14, 2023.

Lin, Z., Miao, Y., Zhang, Q., Yang, F., Zhu, Y., Li, C., Maleki, S., Cao, X., Shang, N., Yang, Y., Xu, W., Yang, M., Zhang, L., and Zhou, L. nnScaler: Constraint-Guided Parallelization Plan Generation for Deep Learning Training. In *Proc. OSDI '24*, pp. 347–363, 2024.

Liu, D., Yan, Z., Yao, X., Liu, T., Korthikanti, V., Wu, E., Fan, S., Deng, G., Bai, H., Chang, J., Aithal, A., Andersch, M., Shoeybi, M., Yao, J., Zhou, C., Wu, D., Li, X., and Yang, J. MoE Parallel Folding: Heterogeneous Parallelism Mappings for Efficient Large-Scale MoE Model Training with Megatron Core. *arXiv preprint arXiv:2504.14960*, 2025.

Liu, G., Miao, Y., Lin, Z., Shi, X., Maleki, S., Yang, F., Bao, Y., and Wang, S. Aceso: Efficient Parallel DNN Training through Iterative Bottleneck Alleviation. In *Proc. EuroSys '24*, pp. 163–181, 2024.

Llama Team, AI @ Meta. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783*, 2024.

Ma, Z., Wang, G., Xie, X., Chen, Y., Du, H., Li, B., Sun, Y., Liu, W., Chen, K., and Li, Y. TREX: Automating LLM Fine-tuning via Agent-Driven Tree-based Exploration. *arXiv preprint arXiv:2604.14116*, 2026.

Miao, X., Wang, Y., Jiang, Y., Shi, C., Nie, X., Zhang, H., and Cui, B. Galvatron: Efficient Transformer Training over Multiple GPUs Using Automatic Parallelism. *Proc. VLDB Endowment*, 16(3):470–479, 2022.

MiniMax. MiniMax-M2 [model card]. Hugging Face, MiniMaxAI/MiniMax-M2, revision 757303d4, 2025. URL https://huggingface.co/MiniMaxAI/MiniMax-M2.

Mistral AI. Cheaper, Better, Faster, Stronger [Mixtral 8x22B release], 2024. URL https://mistral.ai/news/mixtral-8x22b.

Narayanan, D., Shoeybi, M., Casper, J., LeGresley, P., Patwary, M., Korthikanti, V., Vainbrand, D., Kashinkunti, P., Bernauer, J., Catanzaro, B., Phanishayee, A., and Zaharia, M. Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. In *Proc. SC '21*, pp. 1–15, 2021.

Novikov, A., Vũ, N., Eisenberger, M., Dupont, E., Huang, P.-S., Wagner, A. Z., Shirobokov, S., Kozlovskii, B., Ruiz, F. J. R., Mehrabian, A., Kumar, M. P., See, A., Chaudhuri, S., Holland, G., Davies, A., Nowozin, S., Kohli, P., and Balog, M. AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery. *arXiv preprint arXiv:2506.13131*, 2025.

NVIDIA. Nemotron 3 Nano: Open, Efficient Mixture-of-Experts Hybrid Mamba-Transformer Model for Agentic Reasoning. *arXiv preprint arXiv:2512.20848*, 2025.

NVIDIA. Exemplar Performance (formerly DGX Cloud Benchmarking Recipes). README section Peak Theoretical Throughput, commit 8e9a5c8, 2026a. URL https://github.com/NVIDIA/exemplar-performance.

NVIDIA. Megatron-Bridge. Commit c3c974d: performance summary archive (NeMo 25.11, DGX-B200, Llama-3.1-405B), GLM-4.5 recipe `src/megatron/bridge/recipes/glm/h100/glm45.py`, convergence check `scripts/performance/utils/evaluate.py` and agent skills `skills/`, 2026b. URL https://github.com/NVIDIA-NeMo/Megatron-Bridge/tree/c3c974d4f2e315db8db01945acab6555049c40d7.

NVIDIA. NeMo AutoConfigurator. NeMo v2.7.3 (the last release that ships it), `nemo/collections/llm/tools/auto_configurator`, 2026c. URL https://github.com/NVIDIA-NeMo/Speech/tree/v2.7.3/nemo/collections/llm/tools/auto_configurator.

NVIDIA. NeMo AutoModel. Container 26.08.00, commit 922e83e4; performance summary `docs/performance-summary.mdx`, 2026d. URL https://github.com/NVIDIA-NeMo/Automodel.

OpenAI. gpt-oss-120b & gpt-oss-20b Model Card. *arXiv preprint arXiv:2508.10925*, 2025.

Penedo, G., Kydlíček, H., Ben Allal, L., Lozhkov, A., Mitchell, M., Raffel, C., Von Werra, L., and Wolf, T. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. In *NeurIPS 2024 Datasets and Benchmarks Track*, pp. 30811–30849, 2024.

Qwen Team. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388*, 2025.

RadixArk. Miles v0.1: Production-Level Post-Training. *arXiv preprint arXiv:2609.08368*, 2026.

Rae, J. W., Potapenko, A., Jayakumar, S. M., Hillier, C., and Lillicrap, T. P. Compressive Transformers for Long-Range Sequence Modelling. In *Proc. ICLR '20*, 2020.

Rajbhandari, S., Li, C., Yao, Z., Zhang, M., Aminabadi, R. Y., Awan, A. A., Rasley, J., and He, Y. DeepSpeed-MoE: Advancing Mixture-of-Experts Inference and Training to Power Next-Generation AI Scale. In *Proc. ICML '22*, pp. 18332–18346, 2022.

Ren, J., Rajbhandari, S., Aminabadi, R. Y., Ruwase, O., Yang, S., Zhang, M., Li, D., and He, Y. ZeRO-Offload: Democratizing Billion-Scale Model Training. In *Proc. USENIX ATC '21*, pp. 551–564, 2021.

Sarkar, A., Ghosh, S., Tallent, N., Chadha, A., Roosta, T. G., and Jannesari, A. Rudder: Steering Prefetching in Distributed GNN Training Using LLM Agents. In *Proc. ICS '26*, pp. 353–366, 2026.

Shoeybi, M., Patwary, M., Puri, R., LeGresley, P., Casper, J., and Catanzaro, B. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

Shong, J., Ding, Y., Jiang, Y., Jing, L., Chen, H., Zhang, G., Akella, A., and Lai, F. AutoScout: Structured Optimization for Automating ML System Configuration. *arXiv preprint arXiv:2603.11603*, 2026.

Smit, A., Grinsztajn, N., Duckworth, P., Barrett, T. D., and Pretorius, A. Should We Be Going MAD? A Look at Multi-Agent Debate Strategies for LLMs. In *Proc. ICML '24*, pp. 45883–45905, 2024.

Strati, F., Zhang, Z., Manos, G., Sánchez Périz, I., Hu, Q., Chen, T., Buzcu, B., Han, S., Delgado, P., and Klimovic, A. Sailor: Automating Distributed Training over Dynamic, Heterogeneous, and Geo-distributed Clusters. In *Proc. SOSP '25*, pp. 204–220, 2025.

Um, T., Oh, B., Kang, M., Lee, W.-Y., Kim, G., Kim, D., Kim, Y., Muzzammil, M., and Jeon, M. Metis: Fast Automatic Distributed Training on Heterogeneous GPUs. In *Proc. USENIX ATC '24*, pp. 563–578, 2024.

Varadharajan, Y. S. V., Hu, B., Agarwal, S., and Akella, A. CUCo: An Agentic Framework for Compute and Communication Co-design. *arXiv preprint arXiv:2603.02376*, 2026.

Wang, Q., Wang, Z., Su, Y., Tong, H., and Song, Y. Rethinking the Bounds of LLM Reasoning: Are Multi-Agent Discussions the Key? In *Proc. ACL '24*, pp. 6106–6131, 2024.

Wang, X., Li, Q., Xu, Y., Lu, G., Li, D., Chen, L., Zhou, H., Zheng, L., Zhang, S., Zhu, Y., Liu, Y., Zhang, P., Qian, K., He, K., Gao, J., Zhai, E., Cai, D., and Fu, B. SimAI: Unifying Architecture Design and Performance Tuning for Large-Scale Large Language Model Training with Scalability and Precision. In *Proc. NSDI '25*, pp. 541–558, 2025.

Wen, W., Zhu, Q., Chu, W., Chen, W.-Y., and Yang, J. CubicML: Automated ML for Large ML Systems Co-design with ML Prediction of Performance. *arXiv preprint arXiv:2409.04585*, 2024.

Yan, Z. Megatron MoE Model Zoo: Mixtral-8x22B benchmarking recipe. `best_practice/mixtral/run_mixtral_8x22B.sh`, commit 38ce2ed, 2026. URL https://github.com/yanring/Megatron-MoE-ModelZoo.

Yuan, Q. and Zhao, J. Searching for Plans You Can Actually Build: A Realizability-Aware Full-Space Optimizer for MoE Training and Serving. *arXiv preprint arXiv:2607.18631*, 2026.

Zhang, X., Zhao, H., Xiao, W., Jia, X., Xu, F., Li, Y., Lin, W., and Liu, F. Rubick: Exploiting Job Reconfigurability for Deep Learning Cluster Scheduling. In *Proc. MLSys '25*, 2025.

Zhao, C., Zhou, S., Zhang, L., Deng, C., Xu, Z., Liu, Y., Yu, K., Li, J., and Zhao, L. DeepEP: An Efficient Expert-Parallel Communication Library. GitHub, v1.2.1, 2025. URL https://github.com/deepseek-ai/DeepEP.

Zheng, L., Li, Z., Zhang, H., Zhuang, Y., Chen, Z., Huang, Y., Wang, Y., Xu, Y., Zhuo, D., Xing, E. P., Gonzalez, J. E., and Stoica, I. Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning. In *Proc. OSDI '22*, pp. 559–578, 2022.

Zhipu AI. GLM-4.7-Flash. Hugging Face model card, revision 7dd2089, 2026. URL https://huggingface.co/zai-org/GLM-4.7-Flash.

Zhu, Z., Giannoula, C., Andoorveedu, M., Su, Q., Mangalam, K., Zheng, B., and Pekhimenko, G. Mist: Efficient Distributed Training of Large Language Models via Memory-Parallelism Co-Optimization. In *Proc. EuroSys '25*, pp. 1298–1316, 2025a.

Zhu, Z., Xie, C., Lv, X., and slime Contributors. slime: An LLM Post-Training Framework for RL Scaling. GitHub, commit a897e1f (version 0.3.0), 2025b. URL https://github.com/THUDM/slime.

AI USAGE DISCLOSURE

ATTUNE is itself an AI system, so generative AI is part of the work under study. Every ATTUNE campaign, its ablation arms and the OptiCo baseline ran Claude Opus 4.8 through Claude Code (v2.1.286), as described in Sections 2–3; the campaigns of the reproduced set (Section 3.1) ran Claude Opus 5.5 through Claude Code (v2.1.305–306). The catalog check in Section 6.2 queried Claude Opus 5.5. Beyond the system under study, we used Claude Code with Claude Opus models as follows.

Ideation and experiment design. The authors conceived ATTUNE, designed its architecture and correctness gate, and set the evaluation protocol, budgets and decision rules. AI-simulated reviewer reads were used to find weaknesses in the design and evaluation; the authors decided which to address.

Code. Much of ATTUNE's harness, skill scripts, correctness tools, baseline adapters, and analysis and plotting code was written with Claude Code from the authors' specifications. The authors reviewed and tested every change, and an independent agent also reviewed changes to measurement and scoring.

Experiments and analysis. Agents ran post-hoc analyses under the authors' direction: the re-measurement sensitivity check, the transcript and ledger analyses, and the catalog audit. The authors checked their results against the raw records.

Data. No training data was generated by a model. All rows come from public corpora, processed by scripts.

Literature search. Agents with web access assisted the literature search. References were checked against their canonical records with agent-based checks and CiteCheck.

Writing. Much of the text was drafted and revised with Claude Code from the authors' outlines and decisions. The authors wrote some sections and sentences themselves and revised others. The authors decided every claim and every edit, and take responsibility for the content.

A Workloads, baselines and datasets

Each model trains at its public recipe's workload shape, on real text, built once as a frozen dataset with a manifest (Table A1). Rows are packed to exactly the recipe's sequence length. NVIDIA's published Llama-3.1-405B figure uses mock tokens, and the Mixtral-8x22B figure an unreleased dataset. Mixtral-8x22B's published 445 TFLOP/s comes from the recipe's dropless run with its own load-balancing loss. The recipe lists a forced-balancing run separately, at 465 TFLOP/s. Documents are joined with the tokenizer's end-of-sequence token, with no beginning-of-sequence token, and loss falls on all tokens.

Gutenberg books are drawn from the PG-19 train list (Rae et al., 2020) and FineWeb-Edu rows from its 10B-token sample (Penedo et al., 2024), less documents from a list of excluded sites.

The data loader reads the first (steps $\times$ global batch) rows in manifest order and verifies each row's hash, so runs on a model, by any tuner, see the same rows in the same order. A fidelity check runs each model in our stack and the released one on the same rows: losses agree within 1%, and for the MoE models per-layer expert choices agree on at least 95% of assignments.

Benchmark workload. The benchmark of Section 4 uses 8,192-token rows of SFT conversations from the Tulu 3 SFT mixture (Lambert et al., 2025), rendered with each model's chat template, at global batch 128 in BF16. Its rows are validated in the same way as the datasets above. The Tulu 3 SFT mixture is released as a collection under ODC-By 1.0; some of its subsets carry their own licenses, one of them non-commercial (No Robots, CC-BY-NC-4.0).

Stack fixes. Six stack fixes apply to all tuners: (1) a fix for slime's re-entry bug under data-parallel overlap, with Megatron's FP8 parameter-sync backport, (2) the userbuffers bootstrap for tensor-parallel communication overlap, (3) skipping slime's per-step copy of the weights to host memory, which only reinforcement-learning rollouts need, (4) fixed pipeline-communication shapes when microbatches are single fixed-length rows, (5) garbage collection every 50 steps instead of every step, and (6) restoring NCCL's default 4 MiB buffer size. Each fix passes a test showing that it leaves the loss unchanged within run-to-run noise.

Changes from the public recipes. Table A2 lists each model's changes. The models run under slime's training loop with a pretokenized data feed and the stack fixes above.

B Correctness gate

Configuration check. The objective class of Section 2.2.4 names 182 arguments. Each model's lists also freeze some settings: Mixtral-8x22B and GLM-4.5 keep one row per microbatch, since more rows per microbatch would change their load-balancing loss; Llama-3.1-405B keeps the published FP8 recipe; and MiniMax-M2 keeps its auxiliary-loss-free routing and constant learning rate. A *component test* runs the default path, the candidate and an FP32 reference on identical inputs at the model's shapes, and passes if routing is identical and the candidate's error against the reference is at most about twice the default's.

Loss check. Step 0 precedes any update, so a profiled run, a longer run and a benchmark run of the same recipe give the same step-0 loss, and any run of a recipe can check it. The 5% tolerance is the per-step tolerance that Megatron-Bridge's performance pipeline applies to losses between 0.5 and 2.0. Among the 328 ablation runs, the largest legitimate shift is 1.7% on GLM-4.7-Flash and 0.02% on Nemotron-3-Nano-30B-A3B; Table B2 gives the other runs.

Loss curve, for review. The gate passes the candidate's loss curve and the baseline's to Megatron-Bridge's `validate_convergence`, vendored unmodified (Table C1), with the thresholds of Megatron-Bridge's own performance pipeline, and lists the result in the gate report. The result is advisory: on GLM-4.7-Flash, whose loss spikes

Table A1. Frozen datasets, one per model. Rows × tokens gives the number of packed rows and the tokens per row, which equals the recipe's sequence length. FineWeb-Edu is released under ODC-By 1.0; the Gutenberg texts are in the public domain.

Model	Source	Rows × tokens
Llama-3.1-405B	Project Gutenberg, PG-19 book list (mirror texts); Llama-3.1 tokenizer	6,400 × 8,192
Mixtral-8x22B	Project Gutenberg, PG-19 book list (mirror texts); Mixtral tokenizer	64,000 × 4,096
GLM-4.5	`HuggingFaceFW/fineweb-edu`, `sample/10BT` shards 000–002; GLM-4.5 tokenizer	225,280 × 4,096
MiniMax-M2	`HuggingFaceFW/fineweb-edu`, `sample/10BT` shards 000–002; MiniMax-M2 tokenizer	28,160 × 8,192

Table A2. Changes from each public recipe that affect throughput or the baseline; the baseline every tuner starts from includes them. All models read the datasets of Table A1 and start from released weights.

Model	Changes from the public recipe
Llama-3.1-405B, 128×B200	Per-token loss; vocabulary unpadded to match the converted checkpoint; data-parallel and tensor-parallel communication overlap kept on as published, through stack fixes and a gloo bootstrap; learning rate scaled by 1/30 for the released weights
Mixtral-8x22B, 128×H100	RoPE base 10^6 as in the released model, which the recipe omits; selective recomputation of the expert activation to fit in memory (the recipe recomputes nothing); userbuffers bootstrap over gloo
GLM-4.5, 128×B200	Multi-token-prediction layer removed; B200 instead of the recipe's H100, with the published layout unchanged; data-parallel overlap kept on as published, through a stack fix
MiniMax-M2, 64×B200	Supervised training on fixed 8,192-token rows of FineWeb-Edu instead of the recipe's reinforcement-learning loop; 64 GPUs instead of 128, with tensor, pipeline, expert-tensor and context parallelism kept; released FP8 weights dequantized to BF16; expert parallelism 4→8 to fit in memory; QK-norm weight gradients averaged rather than summed over tensor-parallel ranks

Table A3. Public recipes and their workloads, in tokens/s/GPU unless noted, with 8 GPUs per node (B200: 180 GB and H100: 80 GB of HBM per GPU). "Published" is the throughput the recipe's authors report (NVIDIA, 2026b; Yan, 2026; NVIDIA, 2026d); Appendix D defines MFU. For the reproduced set, "Our stack" is the unchanged recipe in NVIDIA's container (26.08, AutoModel 922e83e4) on our cluster; for the ported set it is the public recipe on slime and Megatron-LM with the changes of Appendix A. Seq: sequence length; GBS: global batch size in rows; –: none published.

Model	Type	GPUs	Public recipe and workload	Published	Our stack
Reproduced on NeMo AutoModel					
Qwen3-30B-A3B	MoE (30B, 3B active)	8×H100	AutoModel benchmark; mock tokens, balanced routing, random init; seq 4K, GBS 512, BF16	12,040 (MFU 28%)	11,963.1
GPT-OSS-20B	MoE (21B, 3.6B active)	8×H100	AutoModel benchmark; mock tokens, balanced routing, random init; seq 4K, GBS 256, BF16	13,058 (MFU 30%)	12,970.3
Nemotron-3-Nano-30B-A3B	hybrid Mamba MoE (30B, 3.5B active)	8×H100	AutoModel benchmark; mock tokens, balanced routing, random init; seq 4K, GBS 512, BF16	16,789 (MFU 34%)	16,538.1
Llama-3.1-70B	dense (70B)	64×H100	AutoModel pretrain benchmark; mock tokens, released weights; seq 8K, GBS 128, BF16	866.8 (MFU 39%)	801.2
Ported to slime and Megatron-LM					
Llama-3.1-405B	dense (405B)	128×B200	Megatron-Bridge pretrain; seq 8K, GBS 64, FP8 current scaling	661 (MFU 37%)	394.9
Mixtral-8x22B	MoE (141B)	128×H100	Megatron-Core MoE model zoo; seq 4K, GBS 256, BF16	445 TFLOP/s (MFU 45%)	259.7 TFLOP/s
GLM-4.5	MoE (355B, 32B active)	128×B200	Megatron-Bridge pretrain, written for H100; seq 4K, GBS 2,048, BF16	–	723.6
MiniMax-M2	MoE (230B, 10B active)	64×B200	slime recipe, written for 128 GPUs; seq 8K, GBS 256, BF16	–	895.2

Table B1. Known stack defects, refused by the configuration check for every tuner on the listed models. On Mixtral-8x22B, expert-tensor parallelism 8 with expert parallelism 1 changes training as it does on GLM-4.5; expert-tensor parallelism 4 with expert parallelism 1 is refused on Mixtral-8x22B because of the GLM-4.5 result.

Known stack defect	How it shows	Refused on
Fused routing (a same-math swap)	breaks exact BF16 ties differently and routes some tokens to other experts; fails its component test	Mixtral-8x22B, GLM-4.5, MiniMax-M2, GLM-4.7-Flash
Gradient-accumulation fusion off (a same-math swap)	adds a BF16 rounding where main gradients are FP32; fails its component test	Mixtral-8x22B, GLM-4.5, MiniMax-M2
Expert parallelism 1 with expert-tensor parallelism of 4 or more	silently changes training: step 0 matches the baseline, and the loss departs from it after the first update	Mixtral-8x22B, GLM-4.5, MiniMax-M2
Context parallelism 8 with virtual stages	hangs at step 0	Mixtral-8x22B
Context parallelism above 4	a step takes about 1,800 s	GLM-4.5
Tensor parallelism 16	cannot run	MiniMax-M2
More than 32 rows per microbatch	cannot run: Transformer Engine's RMSNorm overflows on the tensor-parallel-gathered query, above 2^{31} elements	MiniMax-M2
Pipeline parallelism 32	halves the load-balancing loss while the language-model loss stays identical	GLM-4.5
Uneven pipeline split	loads another layer's weights into the Mamba layers, raising the step-0 loss by 215% to 768%	Nemotron-3-Nano-30B-A3B

during its first ten steps, 151 of the 702 ordered pairs of its 27 baseline-recipe runs fail the check against each other, so a rule on the curve would reject correct recipes. Gating also on the loss curve, under any of three stricter rules, keeps the order of the ablation arms of Section 3.4 on both models and leaves full ATTUNE's means and every Nemotron-3-Nano mean unchanged.

Long run. We trained the shipped recipe of ATTUNE's MiniMax-M2 campaign 1 (Table D3) twice and its public recipe once, for 1,000 steps each, on the evaluation workload's rows and seed at a constant learning rate of 1e-5, ten times the workload's; each run makes about nine passes over the rows. Megatron-Bridge's convergence check passes for the shipped recipe against the public recipe over all 1,000 steps, with a correlation of 0.9995, and the mean relative loss gap over steps 900–999 is 0.89%, against 0.93% between the two runs of the shipped recipe. Over steps 10–999 the shipped recipe is 2.66 times as fast as the public recipe, close to the 2.71 times of the final re-measurement.

C Environment, versions and isolation

All runs on a model use one pinned training image per cluster. Table C1 lists the component versions, which are the same in the B200 and H100 images.

Each node has eight GPUs and 3.2 Tb/s of RDMA networking.

The ATTUNE arms receive their skill and a description of the public recipe, including its source. Each campaign runs in a fresh sandbox that holds only its skill, its campaign directory, the model's correctness rules, checkpoint and data, all read-only except the campaign directory. It reaches the cluster only through a *broker*, a proxy that accepts only that campaign's jobs, checks each job specification, applies the stack fixes of Appendix A and logs all requests. Through it the agent can also read files inside the training image, by running a command in the campaign's own pod.

Isolation audit. In the 45 evaluation campaigns of Tables D2 and D3, the sandbox and broker denied every request for a file or cluster object outside the campaign, and no tuner opened any of the ten recipe files for the evaluation models that the training images contain.

D Budgets, re-measurement and per-campaign results

Charging. Every launch is charged, including failed and profiled launches and correctness checks, even when it overshoots the budget, and the number of launches is not limited. A launch that fails on infrastructure is charged to an evaluation reserve; a retry of the same recipe on other nodes is charged to the tuner, and if it also fails the recipe counts as failed. Every tuner runs at most three jobs at a time. ATTUNE and OptiCo may stop before the budget is spent, while Bayesian optimization and random search run to it.

Table B2. The correctness gate's validation per fault and model, on runs outside the evaluation campaigns. The step-0 loss shift is relative to the model's baseline. Counts in parentheses are fault-free runs. "Refused" means the configuration check rejects the candidate before launch, and the step-0 shifts of refused configurations come from running them outside any campaign; "passes" means the configuration check cannot see the fault. EP, ETP, PP: expert, expert-tensor and pipeline parallelism.

Run	Models	Configuration check	Step-0 loss shift	Gate
Fault-free: replicates, single placement or packing changes, runs of the public recipe and legitimate recipes whose step-0 numerics differ from the baseline	GLM-4.5 (27), MiniMax-M2 (31), Mixtral-8x22B (28), Llama-3.1-405B (4); development: GPT-OSS-20B (32), GPT-OSS-120B (10)	admitted	at most 1.98%	**132 of 132 pass**
EP 1 with ETP of 4 or more (a real stack defect that silently changes training)	GLM-4.5, Mixtral-8x22B, MiniMax-M2	refused	at most 0.02%	**rejected**
PP 32, which halves the load-balancing loss while the language-model loss stays identical (a real stack defect)	GLM-4.5	refused	0.00%	**rejected**
Argument faults: capacity factor 1, learning rate $\times 10$, global batch changed	development: Qwen3.5-122B-A10B	refused	+7.6%, -0.02%, +0.2%	**rejected**
Argument faults: BF16 gradient reduction, per-token loss off, auxiliary-loss coefficient changed	development: Qwen3.5-122B-A10B	refused	-0.04%, +0.5%, -0.01%	**rejected**
Labels offset by one token	GLM-4.5, MiniMax-M2, Llama-3.1-405B; development: GPT-OSS-120B	passes	+190% to +872%	**rejected**
Attention sinks ignored	development: GPT-OSS-120B	passes	+151%	**rejected**
One gradient bucket's reduction skipped	GLM-4.5, MiniMax-M2, Mixtral-8x22B, Llama-3.1-405B; development: GPT-OSS-120B	passes	0.00%	missed
Load-balancing loss zeroed	GLM-4.5, Mixtral-8x22B	passes	0.00%	missed
Loss mask shifted by one token, count kept	development: GPT-OSS-20B	passes	+2.0%	missed
QK-norm weight gradients summed instead of averaged over tensor-parallel ranks (a real stack defect)	MiniMax-M2	passes	0.00%	missed

Table C1. Software in the pinned training images and the tuners' sandbox.

Component	Version
OS	Ubuntu 24.04
Python	3.12.3 (training image)
PyTorch	2.11.0 (CUDA 12.9 build)
CUDA	12.9
NCCL / libfabric NCCL plugin	2.28.9 / 1.20.0
Megatron-LM	commit `1dcf0da`
slime	0.3.0: THUDM/slime `a897e1f` plus 8 commits that do not change the training step
Transformer Engine	2.10
DeepEP	1.2.1
Megatron-Bridge convergence check	commit `c3c974d`, vendored
Claude Code (agentic tuners)	2.1.286
OptiCo (baseline)	OptiCo-public commit `e5efc74`
Optuna (black-box searches)	5.0.0

The band has a 3% floor because five runs in one time window cannot resolve smaller differences, and a smaller gain rarely repays validating a new recipe.

Final stage. After all campaigns on a model, the baseline and the shipped recipes run twice more in one randomized, interleaved order, on the nodes the scheduler assigns, and a recipe's value is the mean of all its repeats. A recipe got one or two extra repeats when its first two disagreed by more than the band or one ran on a degraded GPU. Minimal-skill campaign 1's shipped recipe was measured in a separate interleaved block and scaled to the final stage by a candidate measured in both.

Ablation campaigns. The 40 campaigns of Table 3 run each job on one 8×H200 node, on miles v0.1.0 (RadixArk, 2026), with 8,192-token rows at global batch 128 and web tools disabled. Each run lasts 20 steps, scored over steps 5–19. Each campaign may spend 20 times its model's mean calibration-run cost, about 45 GPU-hours, and ships by the rule of Section 2.2.4.

Statistics. Beside each mean-rule verdict we report two exact one-sided permutation p-values for the difference of means. The unrestricted test runs over all 20 assignments of the six campaign values to the two arms. The block-stratified test pairs the two arms' same-numbered campaigns into blocks and runs over the 8 assignments that swap values only within a block, so its smallest possible p-value is 0.125.

Efficiency metrics. *Wall-clock* runs from a campaign's first cluster admission to its stop record, excluding intervals in which all its jobs wait in the queue. *Launches* counts search launches, excluding correctness checks. *Failed launches* counts the launches the failure classifier (Appendix E) labels out of memory, hang or crash, with their charged GPU-hours; it excludes refused launches, infrastructure-reserve charges and runs stopped early by the tuner, its pruner or the harness.

Total compute. The evaluation used 56,712 GPU-hours on the cluster, including calibration, bring-up, correctness checks, the final re-measurement and the 1,000-step runs.

TFLOP/s and MFU. Table A3 converts Mixtral-8x22B's throughput to TFLOP/s with Megatron's FLOP count per token, to compare with the published figure. Tables 1 and A3 report MFU = F × throughput ÷ peak, where:

- F, the model FLOPs per token, is 6 × (dense parameters excluding the input embedding + active expert parameters) plus the attention matrix multiplications, counted with causal masking;
- *throughput* is tokens/s/GPU;
- *peak* is the GPU's dense peak at the recipe's precision, as NVIDIA's Exemplar Performance recipes use for MFU (NVIDIA, 2026a): H100 BF16 989 TFLOP/s, B200 BF16 2,250 TFLOP/s and B200 FP8 4,500 TFLOP/s.

Payback.

$$\text{payback days} = \frac{\text{campaign GPU-hours}}{\text{GPUs} \times 24 \times \left(1 - \frac{1}{1+\text{gain}}\right)},$$

with the GPU-hours the campaign actually used. This is the training time after which a run on the tuned recipe has saved the campaign's cost. Payback ignores the agent's LLM cost, about $112 per GLM-4.5 campaign and $58 per MiniMax-M2 campaign at list price by Claude Code's accounting. At its mean campaign cost, ATTUNE pays for itself after 0.9 days on GLM-4.5, 2.0 days on Llama-3.1-405B and 0.2 days on MiniMax-M2. The other tuners take 0.9–1.4 days on MiniMax-M2. On GLM-4.5 random search takes 7.6 days and Bayesian optimization with the pre-filter 14.6 days. On GLM-4.5 payback is omitted for OptiCo, whose

Table D1. Per-launch cost and budgets, from at least five calibration runs per model on distinct node allocations. The calibration mean is the denominator during search; reported gains use the final re-measurement. σ is the calibration runs' relative standard deviation, and each band is max(3%, 3σ). The budget is 30 times the unprofiled launch cost, rounded up to a multiple of 10 GPU-hours.

Model	Calibration mean (tokens/s/GPU)	σ	Band	Unprofiled launch (GPU-h)	Budget per campaign (GPU-h)
Llama-3.1-405B / 128×B200	399.36	1.29%	3.88%	61.38	1,850
Mixtral-8x22B / 128×H100	1,091.69	0.95%	3.00%	24.57	740
GLM-4.5 / 128×B200	728.19	0.49%	3.00%	62.58	1,880
MiniMax-M2 / 64×B200	893.18	0.95%	3.00%	17.58	530

campaigns there all ship the public recipe, and for Bayesian optimization without the pre-filter, whose mean gain stays inside the noise band. These figures assume the measured speedup holds over a full training run. On MiniMax-M2 a 1,000-step run kept it (Appendix B).

Reproduced set. The four NeMo AutoModel campaigns of Table 1 run NVIDIA's benchmark recipes in its 26.08 container (AutoModel 922e83e4; GPT-OSS-20B with upstream fix #3926 for an attention-bias NaN) on IAD H100 nodes, through the same harness, control-plane scripts and correctness gate with an AutoModel stack adapter. Each run keeps the recipe's run length and scored window: 30 iterations scored over 10–29 on one node, and 20 steps scored over 5–19 for Llama-3.1-70B on eight nodes. The real-text checks train from the released weights on SFT rows built with the OpenRecipes builder, with the model's own router and a constant learning rate, and compare the shipped recipe with the public recipe under Megatron-Bridge's convergence check. All four pass. On Qwen3-30B-A3B the losses are identical. On GPT-OSS-20B the shipped recipe deviates from the public recipe by 0.02% over steps 50–99, against 0.03% between two runs of the public recipe, and on Llama-3.1-70B the FlashAttention-3 swap deviates by 0.21% against 0.16% between two SDPA runs. Nemotron-3-Nano's full recipe exceeds GPU memory on real text, so its check runs without selective recomputation, a placement change; its logged losses match the public recipe's at 58 of 60 steps. On real text, GPT-OSS-20B's public recipe also needs PyTorch's expandable_segments allocator setting to fit, which both arms use.

Ablation arms on GLM-4.5. On GLM-4.5 at 128×B200, three more arms run three campaigns each: a *committee* of agents picks each candidate, a catalog written from public tuning guides replaces the one distilled from past campaigns, and the *minimal skill* drops the catalog. Nearly all campaigns find the *intranode-expert recipe*, expert parallelism inside a node with the DeepEP dispatcher (Zhao et al., 2025). The largest lever, the *TP1 layout* with tensor parallelism one at 16 pipeline stages, runs out of memory under the baseline's recomputation, and the two best campaigns, one of full ATTUNE and one of the committee, are the two that retried it with recomputation as rule 5 (Section 2.2.1) directs.

A single agent. The committee is a proposer, a memory-and-validity checker and a profile reader, whose chair picks one candidate per cycle. It is the cheapest arm but less consistent than full ATTUNE: one campaign shipped the TP1 layout, as one of full ATTUNE's did, while another shipped little more than the public recipe.

Catalog contents. The public-guide catalog is written only from public tuning guides (Appendix G). Its arm is the most consistent: all three campaigns shipped the intranode-expert recipe without recomputation. Its one attempt at tensor parallelism one, at eight pipeline stages, ran out of memory and was not retried, so it never reached the TP1 layout. Its guides recommend tensor-parallel communication overlap, which the configuration check refused in these campaigns as a swap without a component test (Section 3.5).

The catalog. The minimal skill keeps the loop, rules and scripts and drops the catalog. It found the intranode-expert recipe in every campaign but not the TP1 layout: each campaign tried tensor parallelism one once, at eight pipeline stages, and did not retry it after that run failed, so full ATTUNE's best campaign is 17% faster than its best.

E BASELINE TUNERS

OptiCo. We run the public OptiCo repository at commit `e5efc74` with its analyzer and prompts unchanged and supply three parts of our own: the LLM (the one ATTUNE uses, at its default sampling temperature rather than OptiCo's 0.7), the slime adapter and the loop driver. The repository's driver runs a fixed 20 cycles, while ours runs OptiCo until the budget is spent or 10 consecutive iterations launch nothing.

- **Workload:** the adapter takes the sequence length, global batch and data from the model's frozen baseline and maps OptiCo's layer-count knobs onto Megatron's pipeline split.

Table D2. Every evaluation campaign on GLM-4.5, in tokens/s/GPU from the final re-measurement. Gain is over the re-measured public recipe (723.6). GPU-hours include correctness checks. Stop: the agent's convergence judgment, the GPU-hour budget or our OptiCo driver's stop after 10 consecutive iterations that launch nothing (no launch). BO: Bayesian optimization; TPE: tree-structured Parzen estimator. Changes are relative to the public recipe. TP, PP, EP, ETP, CP, DP: tensor, pipeline, expert, expert-tensor, context and data parallelism; VP: virtual stages per pipeline rank; first/last-stage layers: transformer layers in the first and last pipeline stages.

Arm	#	Shipped	Gain	GPU-h used	Stop	Shipped changes
ATTUNE	1	885.7	+22.4%	753.6	agent	PP 8→16; EP 16→8; first/last-stage layers 10/10→4/4; DeepEP dispatcher; shared-expert overlap off
ATTUNE	2	1,119.3	+54.7%	626.3	agent	TP 2→1; PP 8→16; EP 16→8; VP off→2; first/last-stage layers 10/10→4/4; full recompute
ATTUNE	3	902.1	+24.7%	627.3	agent	EP 16→8; DeepEP dispatcher; shared-expert overlap off; data-parallel overlaps off
OptiCo	1	723.6	0.0%	1,274.4	no launch	none (public recipe)
OptiCo	2	723.6	0.0%	1,543.8	budget	none (public recipe)
OptiCo	3	723.6	0.0%	1,402.2	no launch	none (public recipe)
BO (TPE)	1	762.8	+5.4%	1,807.8	budget	PP 8→4; even stage split; data-parallel overlaps off
BO (TPE)	2	723.6	0.0%	1,663.7	budget	none (public recipe)
BO (TPE)	3	723.6	0.0%	1,554.1	budget	none (public recipe)
BO + pre-filter	1	765.6	+5.8%	1,676.7	budget	PP 8→4; even stage split; data-parallel overlaps off
BO + pre-filter	2	723.6	0.0%	1,567.0	budget	none (public recipe)
BO + pre-filter	3	766.8	+6.0%	1,841.5	budget	PP 8→4; even stage split; data-parallel overlaps off
Random search	1	723.6	0.0%	1,546.7	budget	none (public recipe)
Random search	2	892.8	+23.4%	1,787.5	budget	TP 2→1; EP 16→8; CP 1→2; VP off→2; no recompute; data-parallel overlaps off
Random search	3	723.6	0.0%	1,729.6	budget	none (public recipe)
Multi-agent committee	1	763.3	+5.5%	662.4	agent	shared-expert overlap off
Multi-agent committee	2	1,126.7	+55.7%	499.1	agent	TP 2→1; PP 8→16; EP 16→8; first/last-stage layers 10/10→4/4; full recompute, 2 layers per checkpoint
Multi-agent committee	3	896.2	+23.8%	650.7	agent	EP 16→8; DeepEP dispatcher; shared-expert overlap off; data-parallel overlaps off
Public-guide catalog	1	931.2	+28.7%	1,284.3	agent	EP 16→8; no recompute; DeepEP dispatcher; shared-expert overlap off; data-parallel overlaps off
Public-guide catalog	2	937.8	+29.6%	725.3	agent	EP 16→8; no recompute; DeepEP dispatcher; shared-expert overlap off; data-parallel overlaps off
Public-guide catalog	3	933.2	+29.0%	600.8	agent	EP 16→8; no recompute; DeepEP dispatcher; shared-expert overlap off; data-parallel overlaps off
Minimal skill	1	959.0	+32.5%	962.0	agent	EP 16→8; no recompute; DeepEP dispatcher; DeepEP SMs 64; shared-expert overlap off; data-parallel overlaps off
Minimal skill	2	938.2	+29.6%	489.5	agent	EP 16→8; no recompute; DeepEP dispatcher; shared-expert overlap off; data-parallel overlaps off
Minimal skill	3	931.0	+28.7%	867.3	agent	EP 16→8; no recompute; DeepEP dispatcher; shared-expert overlap off; data-parallel overlaps off; CUDA_DEVICE_MAX_CONNECTIONS=8

Table D3. Every evaluation campaign on MiniMax-M2, Mixtral-8x22B and Llama-3.1-405B, as in Table D2. Gain is over the model's re-measured public recipe (MiniMax-M2 895.2, Mixtral-8x22B 1,072.4, Llama-3.1-405B 394.9). Embedding and loss accounting: Megatron counts the embedding and the loss as pipeline layers.

Arm	#	Shipped	Gain	GPU-h used	Stop	Shipped changes
MiniMax-M2 / 64×B200						
ATTUNE	1	2,423.1	+170.7%	200.2	agent	TP 2→1; PP 2→4; first/last-stage layers even→15/15; 2 rows per microbatch; data-parallel overlaps on; optimizer offload off
ATTUNE	2	2,235.8	+149.8%	197.3	agent	PP 2→8; first/last-stage layers even→7/7; 4 rows per microbatch; optimizer offload off
ATTUNE	3	2,436.3	+172.2%	213.0	agent	TP 2→1; PP 2→8; VP off→2; first/last-stage layers even→8/6; recompute uniform→block, 2 layers; optimizer offload off
OptiCo	1	1,101.3	+23.0%	301.4	no launch	TP 2→1; data-parallel overlaps on; cross-entropy fusion on
OptiCo	2	1,109.9	+24.0%	384.5	no launch	TP 2→1; data-parallel overlaps on; cross-entropy fusion on
OptiCo	3	1,077.0	+20.3%	473.0	budget	2 rows per microbatch; data-parallel overlaps on; cross-entropy fusion on
BO (TPE)	1	1,651.2	+84.5%	462.9	budget	PP 2→4; VP off→2; first/last-stage layers even→16/14; 2 rows per microbatch; DeepEP dispatcher; optimizer offload off
BO (TPE)	2	998.5	+11.5%	535.7	budget	selective recompute of LayerNorm
BO (TPE)	3	1,285.9	+43.6%	477.8	budget	VP off→31; optimizer offload off
BO + pre-filter	1	1,349.4	+50.7%	467.5	budget	optimizer offload off
BO + pre-filter	2	1,384.1	+54.6%	503.2	budget	PP 2→4; first/last-stage layers even→15/15; optimizer offload off
BO + pre-filter	3	1,441.2	+61.0%	496.9	budget	TP 2→1; PP 2→8; EP 8→4; CP 1→2; first/last-stage layers even→7/7; selective recompute of the MoE layers; DeepEP dispatcher; optimizer offload off
Random search	1	961.8	+7.4%	473.7	budget	PP 2→4; EP 8→2; ETP 1→2; CP 1→2; first/last-stage layers even→15/15; 4 rows per microbatch; DeepEP dispatcher; optimizer offload off
Random search	2	1,001.2	+11.8%	517.7	budget	TP 2→1; PP 2→16; EP 8→2; CP 1→4; VP off→2; first/last-stage layers even→2/4; 2 rows per microbatch; optimizer offload off
Random search	3	1,599.6	+78.7%	470.6	budget	TP 2→1; PP 2→4; EP 8→4; first/last-stage layers even→15/15; optimizer offload off
Mixtral-8x22B / 128×H100						
ATTUNE	1	1,072.4	0.0%	204.8	agent	none (public recipe)
ATTUNE	2	1,072.4	0.0%	194.1	agent	none (public recipe)
ATTUNE	3	1,072.4	0.0%	121.4	agent	none (public recipe)
Llama-3.1-405B / 128×B200						
ATTUNE	1	438.5	+11.1%	317.8	agent	PP 8→16; VP 8→2; DP 2→1
ATTUNE	2	415.3	+5.2%	273.8	agent	VP 8→2; first/last-stage layers 16/14 instead of embedding and loss accounting
ATTUNE	3	411.8	+4.3%	590.0	agent	VP 8→4

Table D4. Ablations on GLM-4.5 / 128×B200, three campaigns per arm. Gain over the public recipe is the mean, with each campaign's in parentheses; the other columns are per-campaign means as in Table 2, and bold marks the best value per column. At GLM-4.5's band of 3.00%, full ATTUNE is ahead of the committee under the mean rule (mean throughput 4.3% higher; exact permutation p = 0.45 unrestricted, 0.375 block-stratified) and of the public-guide arm (3.7%; p = 0.50, 0.50), and ties the minimal skill (2.8%; p = 0.50, 0.50); with three campaigns per arm, none of these differences is significant.

Arm	Gain	GPU-h used	Launches	Failed launches (GPU-h)	Wall-clock
Full ATTUNE	**+33.9%** (+22.4, +54.7, +24.7%)	669	8.3	24% (60)	6.4 h
Multi-agent committee	+28.3% (+5.5, +55.7, +23.8%)	**604**	**6.0**	17% (**30**)	**5.1 h**
Public-guide catalog	+29.1% (+28.7, +29.6, +29.0%)	870	9.3	**11%** (54)	7.0 h
Minimal skill (no lever catalog)	+30.3% (+32.5, +29.6, +28.7%)	773	10.0	13% (41)	6.2 h

Table D5. ATTUNE's recipes in Amazon's post-training jobs, with their GPU counts and the recipes they replaced. Megatron-Bridge, verl and slime recipes are public. Gain is tokens/s/GPU over the replaced recipe on 8,192-token rows at global batch 128; the geometric mean is 1.7×. † Fitted to the GPU count. ‡ Written for 128 GPUs; the ATTUNE recipe runs on 64. Cost uses AWS EC2 on-demand prices in us-east-1 per GPU (H100 $6.88, H200 $7.91, B200 $14.24 per hour), not what Amazon pays.

Recipe	Replaced recipe	Gain	$ per 1M tokens
Qwen3.5-122B, 48×H100	Megatron-Bridge	2.49×	4.86 → 1.95
Qwen3.5-122B, 32×H200	in-house	1.32×	1.24 → 0.94
Qwen3.5-122B, 16×B200	in-house	1.26×	1.94 → 1.54
Qwen3.5-397B, 128×H100	verl	1.25×	7.80 → 6.23
Qwen3.5-397B, 48×H200	verl†	1.06×	7.68 → 7.22
Qwen3.5-397B, 64×B200	verl‡	3.91×	12.00 → 3.07
GLM-5.2, 256×H100	slime	1.07×	30.17 → 28.10
GLM-5.2, 128×H200	slime	2.54×	34.80 → 13.71
GLM-5.2, 64×B200	slime†	2.21×	58.07 → 26.33

- **Schema:** OptiCo's 11 knobs, unchanged. Proposed keys and values are either mapped or rejected before launch.
- **Rejection list:** the microbatch knob where a model freezes one row per microbatch, and any change the model's frozen arguments forbid. At tensor parallelism 1 the adapter removes tensor-parallel communication overlap, as for the black-box searches (Table E1).
- **Co-changes:** the adapter applies each model's required co-changes, such as turning the data-parallel overlaps off on GLM-4.5 when expert data parallelism exceeds 1, so that a schema-valid proposal maps to a valid configuration where one exists. Proposals it still rejects do not count toward the driver's stop.
- **Shared environment:** OptiCo uses the same control plane, correctness gate and broker as the other tuners. The shared failure classifier labels a run a hang when it makes no log progress for 30 minutes after admission.
- **Repeated proposals:** a proposal identical to a configuration the campaign already ran is answered with that run's measurement at no cost, and counts as a cached iteration.
- **Stop reasons:** four of OptiCo's six campaigns ended on the driver's stop after repeatedly proposing configurations they had already run; the other two ran to the budget.

Black-box search. The searches run on GLM-4.5 and MiniMax-M2, each over its frozen space (Table E1). The same space bounds the memory pre-filter's draws.

- **Searched:** the knobs are structural or same-math swaps (the dispatcher, and on MiniMax-M2 the optimizer-offload toggle, which the baseline sets on). On GLM-4.5 optimizer offload is unreachable for all arms: Megatron allows it only with the precision-aware optimizer, a numeric setting that the configuration check freezes and that GLM-4.5's public recipe does not use. The space holds cross-entropy fusion, the data-parallel overlaps and shared-expert overlap at the baseline's values, except that GLM-4.5's DeepEP draws turn shared-expert overlap off. On GLM-4.5 the launcher also removes the data-parallel overlaps when expert data parallelism exceeds 1, because they hang in that setting.
- **Feasibility:** a draw that violates a divisibility or layer-count constraint read from the framework source, or hits a known stack defect (Table B1), scores zero without launching.

Settings. Bayesian optimization uses TPE with 8 startup trials, multivariate sampling and `constant_liar` for three concurrent trials, with seeds 1–3. Random search samples uniformly, with seeds 1–3. Each search takes the baseline as its first trial. A failure of any kind scores zero:

Table E1. Search spaces. On GLM-4.5 DeepEP is offered only within a node, because it cannot initialize across nodes on our network; on MiniMax-M2 it is offered across nodes too, and those draws fail at startup and score zero. The largest virtual-stage values put one layer in each virtual stage, and the light splits give the first, last or both end stages fewer layers. Megatron sets tensor-parallel communication overlap from the tensor-parallel degree, and the launcher removes it at tensor parallelism 1.

Knob	GLM-4.5 (128 GPUs)	MiniMax-M2 (64 GPUs)
Tensor parallelism	1, 2, 4, 8	1, 2, 4, 8
Pipeline parallelism	1, 2, 4, 8, 16	1, 2, 4, 8, 16
Expert parallelism	1, 2, 4, 8, 16, 32 (1 only with expert-tensor parallelism ≤ 2)	1, 2, 4, 8, 16, 32, 64 (1 only with expert-tensor parallelism ≤ 2)
Expert-tensor parallelism	1, 2, 4, 8	1, 2, 4, 8
Virtual stages per rank	off, 2, 4, 23	off, 2, 31
Pipeline split	balanced, light first, light last, light both	balanced, light first, light last, light both
Context parallelism	1, 2, 4	1, 2, 4, 8
Rows per microbatch	1 (frozen)	1, 2, 4, 32
Recompute	none; selective (four module sets); full with 1 or 2 layers per checkpoint	none; selective (four module sets); full with 1 or 2 layers per checkpoint
Dispatcher	all-to-all; DeepEP (within a node, shared-expert overlap off)	all-to-all; DeepEP
Optimizer offload	off (unreachable)	on (baseline) or off

an infeasible draw, a pre-filter rejection, an out-of-memory error, a crash or a hang.

Early abort. Bayesian optimization and random search stop (prune) a run whose throughput falls below 0.90× the leader's after at least four steady measured steps. A pruned trial gets no throughput score. Only these two searches use this rule.

Memory pre-filter. An analytical estimate of peak memory per GPU rejects a draw when the estimate × (1 + margin) exceeds the GPU's memory. Each model's margin is the largest that still admits all configurations that fit in memory in the model's bring-up launches: 5.8% on GLM-4.5 and 11.7% on MiniMax-M2. The baseline trial is exempt from the filter.

F AGENT TRACE: GLM-4.5 ATTUNE CAMPAIGN 2

Table F1 lists the campaign's launches with the agent's diagnosis after each. From the first run's low TFLOP/s and negligible data wait, the agent judged the step bound by communication and kernel launches. No neighbor of the leader beat it by more than the band, so the agent stopped, and the fastest candidate, with two virtual stages, shipped.

G DEVELOPMENT MODELS

ATTUNE was developed on tuning campaigns over these models:

- Qwen3.5-122B-A10B, Qwen3.5-397B-A17B and GLM-5.2-744B-A40B, across H100, H200 and B200;
- Qwen3.5-35B-A3B on H100;
- GPT-OSS-20B and GPT-OSS-120B, on H100;
- ablation testbeds over Qwen3-235B-A22B, Qwen3-30B-A3B, Qwen3-MoE-A0.6B and GPT3-7B.

During those campaigns the agent wrote lessons back to the lever catalog. The frozen catalog states them as general rules, with no per-campaign numbers or summaries. No evaluation model contributed to it.

Catalog audit. Table G1 summarizes two checks of the catalog made after the evaluation (Section 6.2).

Public-guide inputs. The public-guide catalog of Appendix D was written from the public Megatron-LM documentation and the NeMo performance guide by an AI agent with no access to ATTUNE's catalog, the campaign results or the evaluation models.

Table F1. The launches of GLM-4.5 ATTUNE campaign 2 in order, starting from the public recipe (728.2 tokens/s/GPU calibrated). Run names are the agent's labels. Each quote is the agent's diagnosis after the run; in the quotes, SP is sequence parallelism and a2a is all-to-all. Gains up to and including the winner (tp1pp16fr) are over the public recipe, and later gains are over the winner. Values marked ≈ are the agent's estimates for runs it stopped early; the control plane does not score these runs. The shipped recipe re-measured at 1,119.3 tokens/s/GPU in the final stage.

Run	Change → result (tokens/s/GPU)	The agent's reasoning, quoted
pp16ep8	PP 8→16, EP 16→8, first/last-stage layers 10/10→4/4, expert all-to-all inside a node → 707.6, tie	"clearly comm/overhead-bound"; "EP placement is throughput-neutral here"
tp1pp8	TP 2→1 → out of memory	"TP1 (no SP, unsharded activations) doesn't fit at PP8; it needs more PP."
tp1pp16	TP 2→1 at PP 16, EP 8 → out of memory	"memory-infeasible at both PP8 and PP16 with selective recompute"
tp1pp16fr	+ full recomputation → 1,072.7, win (+47.3%)	"our step has many tiny 1-row microbatches → lots of launch idle … full recompute may be ~free here AND it enables the TP1 fit"
tp1pp8fr	back to PP 8, EP 16 → stopped early, ≈668	"the cross-node EP a2a (and DP16 comm) is now exposed"
tp1pp16s53	first/last-stage layers 4/4→5/3 → stopped early, ≈1,083	"the pipeline is not stage-imbalance-bound; it's uniformly comm/launch-bound"
tp1pp16dep	DeepEP dispatcher → stopped early, ≈1,022	"NCCL's intranode alltoall already beats DeepEP's kernels (which also reserve SMs)"
tp1pp16vpp2	two virtual stages → 1,090.3, tie (+1.6%), fastest	"extra per-chunk launches offset the gain on this launch-bound workload"
tp1pp16cp2	CP 1→2 → stopped early, ≈602	"decisive lose, closing the CP axis … All structural neighbors are now measured."
check	check run of tp1pp16vpp2 → passes both gate checks; it ships	–

Table G1. The catalog audit. A lesson is wrong when its own campaign's records contradict it, unsupported when no record supports it, and partly answered when the model's answers miss a condition, scope or failure mode the entry states.

Check	Covers	Result
Lessons checked against the campaigns' own records	155 lessons of the catalog distilled from development campaigns	71 correct and general; 34 rewritten to their general form; 31 wrong; 11 unsupported; 8 resting on one model or campaign
Current model asked, without the skill, about each entry (Claude Opus 5.5, three answers per question)	106 entries of the curated catalog	19 answered fully; 59 partly; 27 contradicted; 1 not known